

# Data-Driven Computer Animation

## Lecture 3.

Motion capture and physically-based  
animation of characters

# Character Animation

- There are three methods
  - Create them manually
  - Use real human / animal motions
  - Use physically based simulation



# Using Real Human (Animal) Motion

Real human (animal) motion is realistic

Much faster and cheaper than manually producing the data



# Capturing human motion

- We use the motion capture device (Mocap)
- There are a few types of Mocaps
  - Optical
  - Magnetic
  - Mechanical
  - Inertial trackers
  - Video-based



# Optical Mocaps

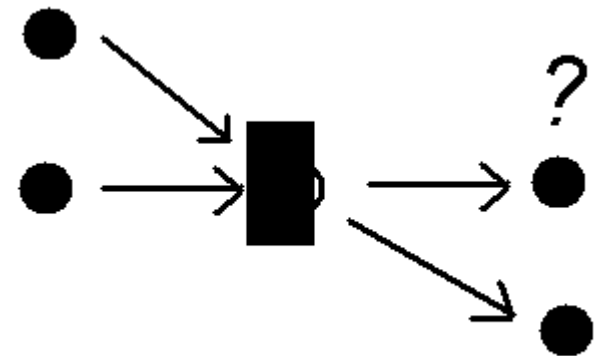
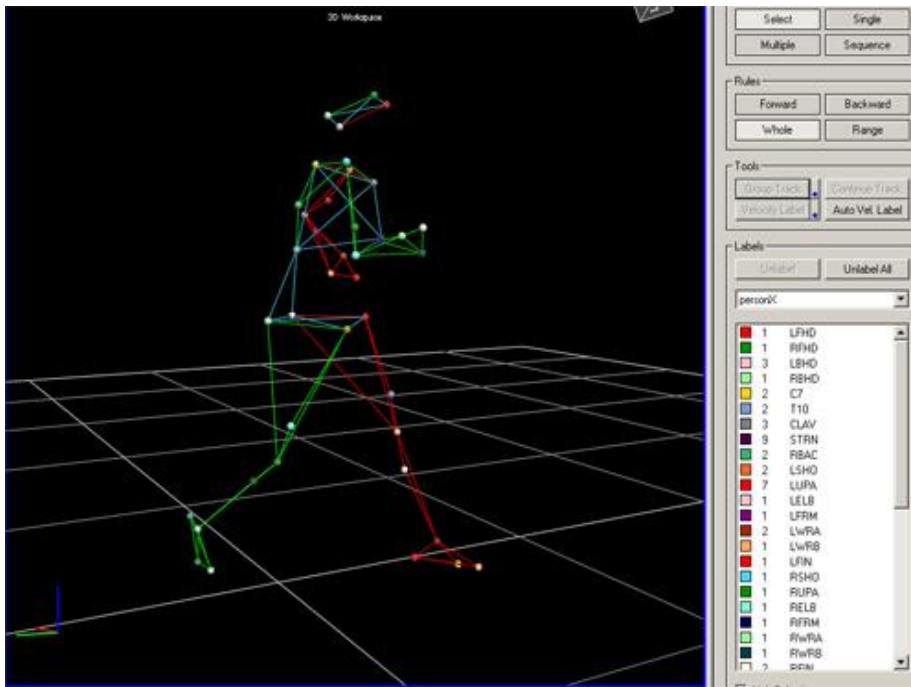
- The actor puts reflective markers on his/her body
- The actor is shot by multiple cameras
- Light source around the camera.
- The light source casts light towards the actor
- The light is reflected by the markers back to the camera
- The 3D location of the markers are computed by stereo vision





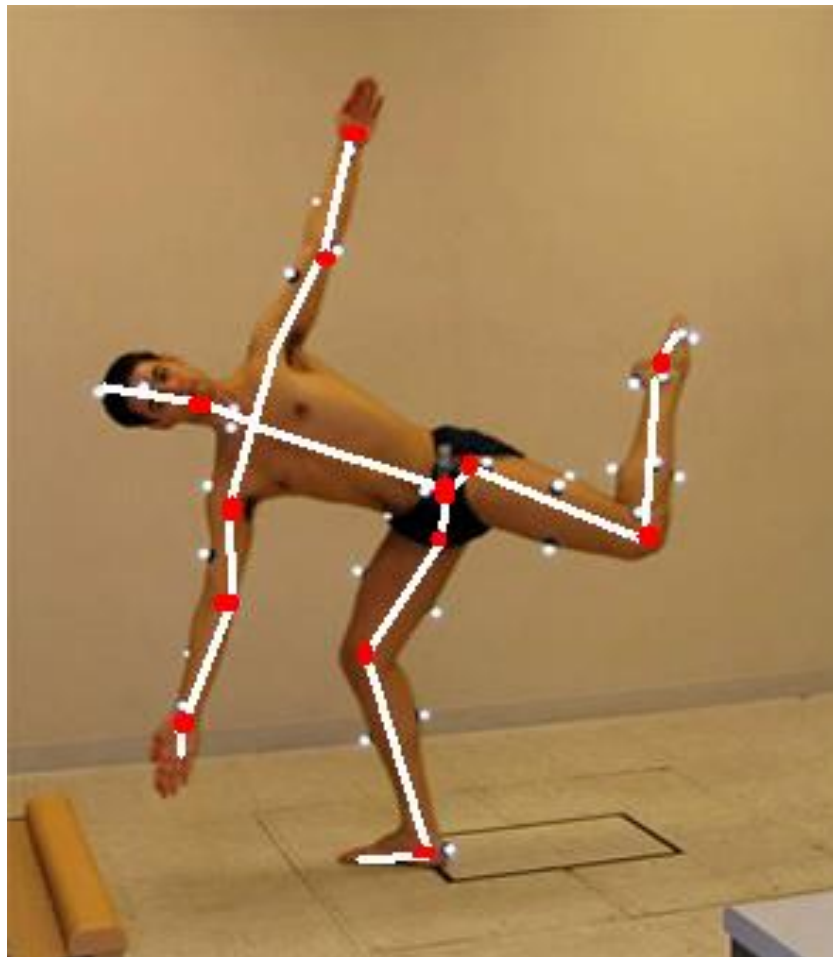
# 1. Labeling and Post-processing

- The markers must be manually labelled first
- Once the markers get occluded, they have to be tagged manually again
  - The same process repeated for the whole motion
- Sometimes the body intersects, and then the system mix them up



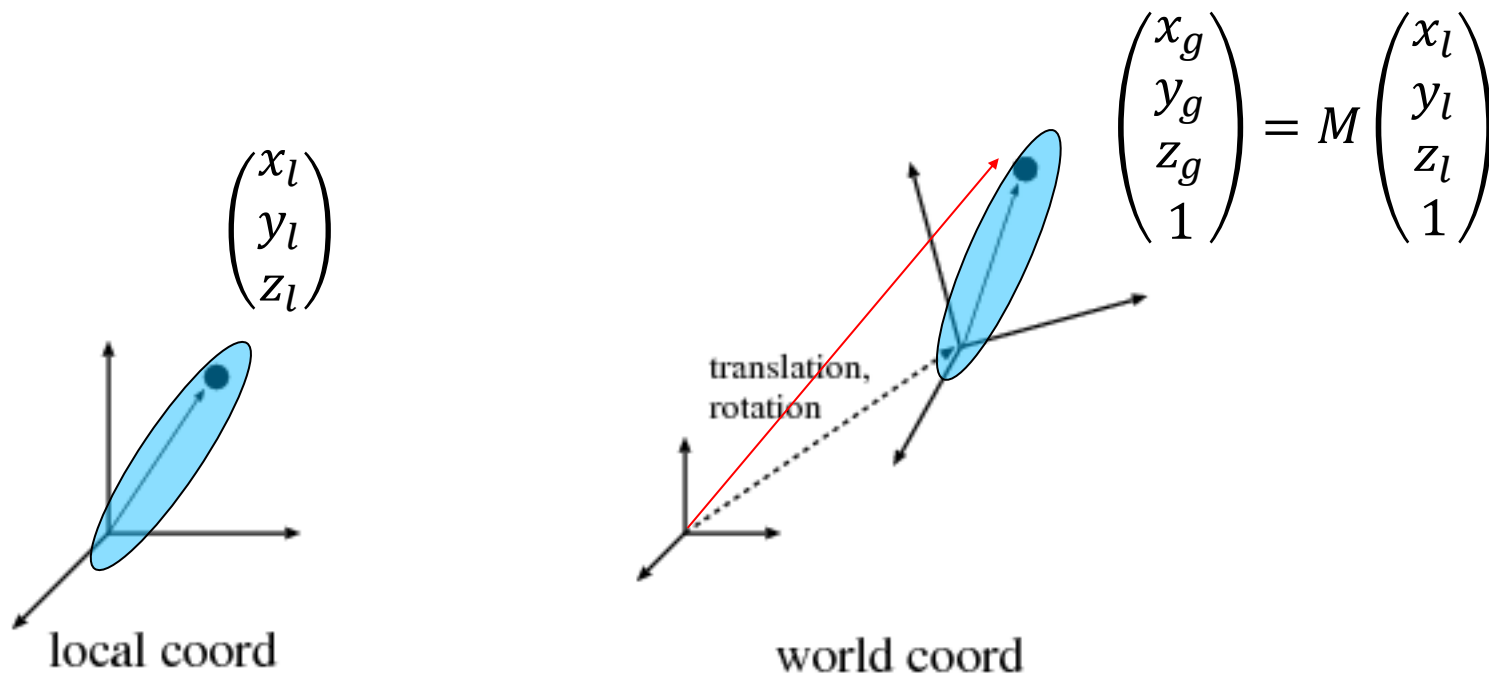
## 2. Computing the center of the joints from the marker positions

- To compute the joint angles of the skeleton



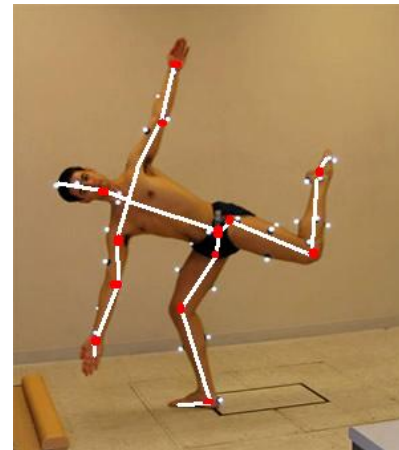
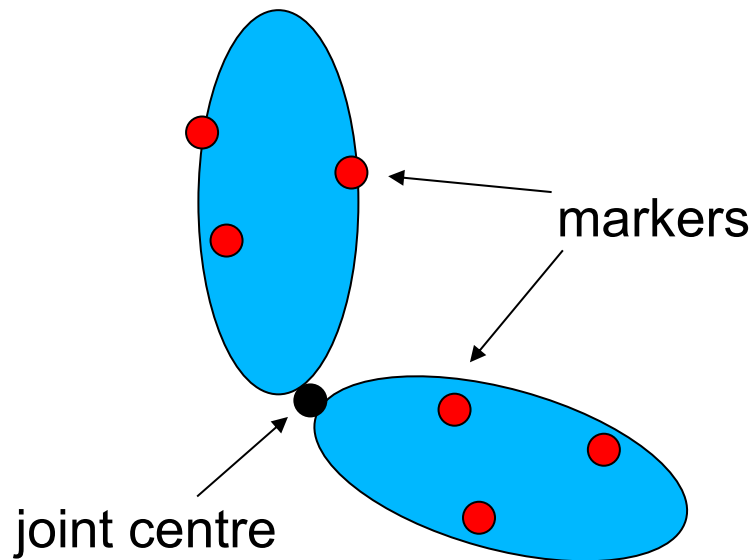
# Background

- A homogeneous transformation matrix  $M$  (4x4) is defined per bone
- It represents the pose of the bone
- It converts local coordinates to world coordinates



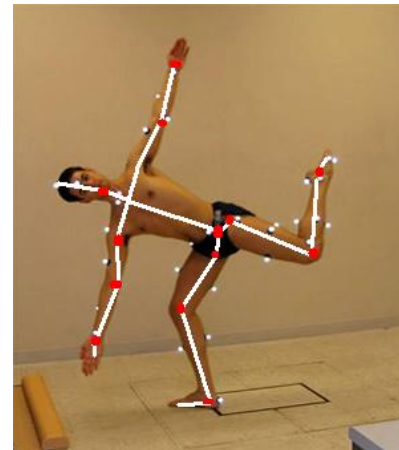
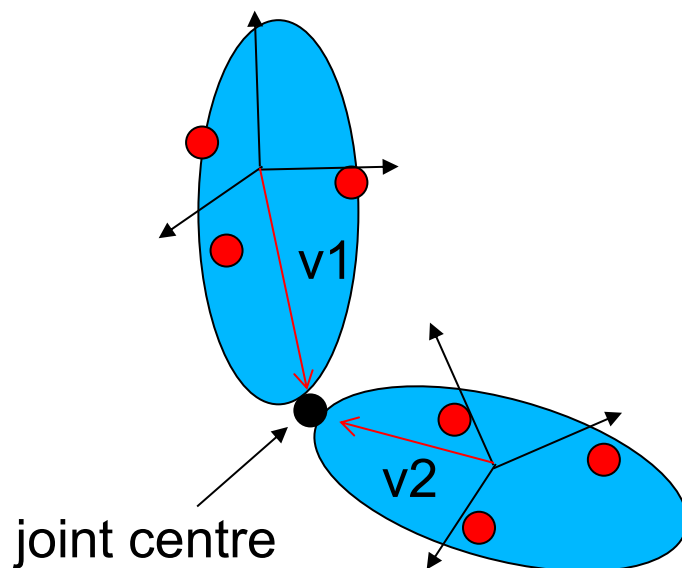
# Predicting the Centre of Joints

- Let's assume we have two adjacent bones connected at a joint
- We assume three markers are attached to each bone
- How can we predict the joint centres with respect to the markers?



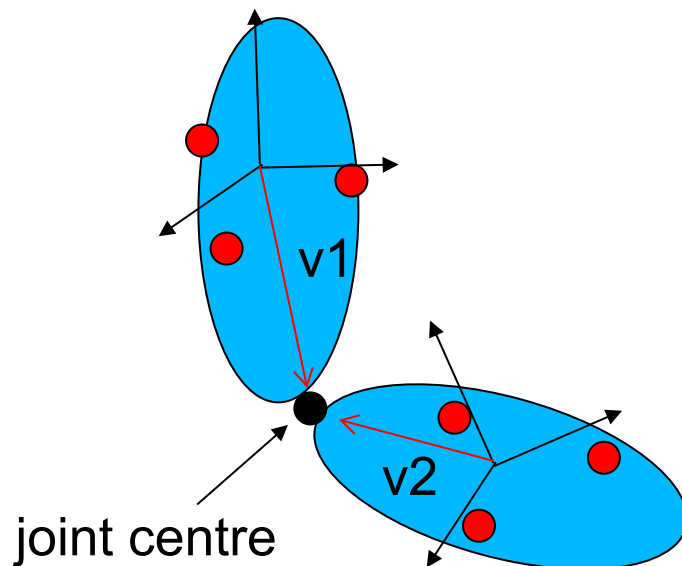
# Predicting the Centre of Joints

- Define the coordinate system of each bone using the markers
- Compute the transformation matrix for each bone in each frame:  $M_1, M_2$
- Define the vector from each bone coordinate to the joint  $(v_1, v_2)$



# Predicting the Centre of Joints

- The equation to compute the joint centre for each frame:  $M_1 v_1$  ,  $M_2 v_2$
- As the positions should be the same,  $M_1 v_1 - M_2 v_2 = 0$
- Decomposing this into the rotation and translation part, it will be  $R_1 v_1 + t_1 = R_2 v_2 + t_2$
- We have this equation for each frame.



$$R_1^1 v_1 + t_1^1 = R_2^1 v_2 + t_2^1$$

$$\vdots$$

$$R_1^N v_1 + t_1^N = R_2^N v_2 + t_2^N$$

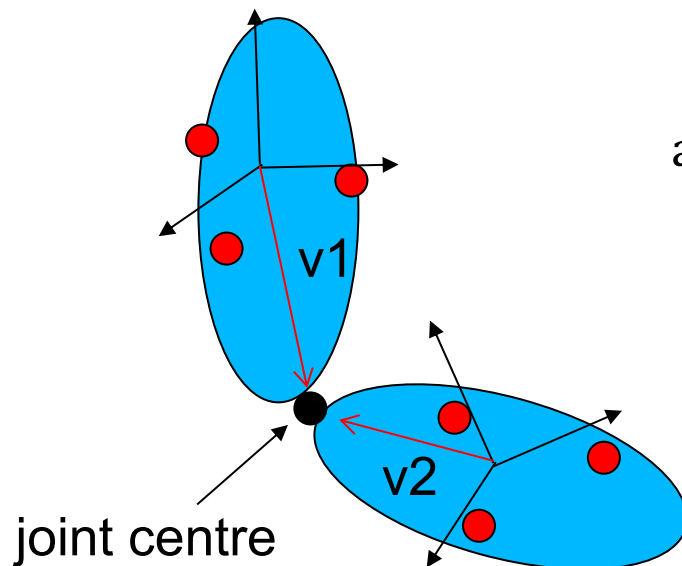
# Predicting the Centre of Joints

We have this equation for each frame.

$$\begin{aligned} R_1^1 v_1 + t_1^1 &= R_2^1 v_2 + t_2^1 \\ &\vdots \\ R_1^N v_1 + t_1^N &= R_2^N v_2 + t_2^N \end{aligned}$$

The only unknowns are  $v_1$   $v_2$

We can solve a least squares problem



$$\arg \min_{v_1, v_2} \sum_{n=1}^N \|R_1^n v_1 + t_1^n - R_2^n v_2 - t_2^n\|^2$$

# Cons and Pros of Optical Mocaps

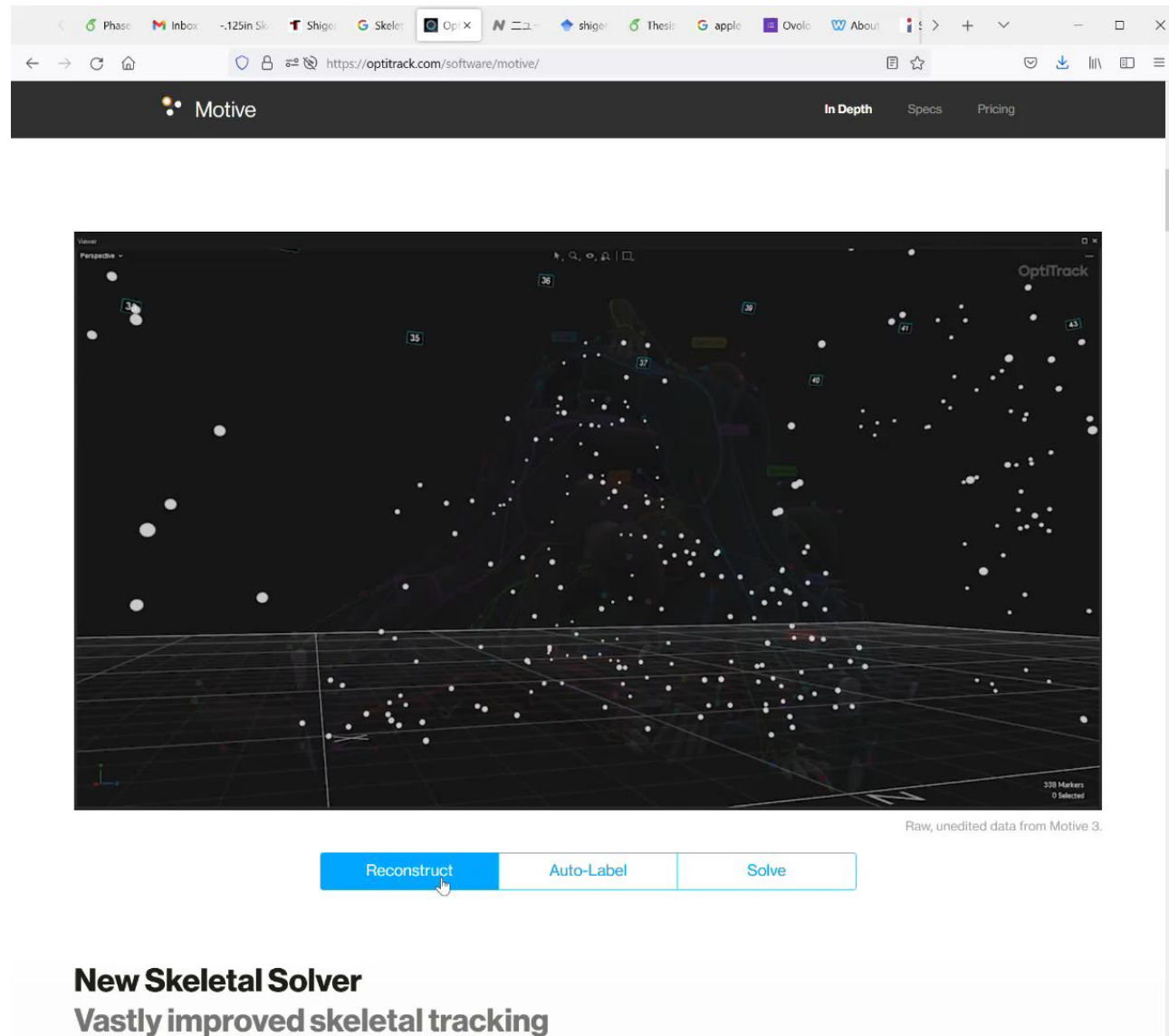
## Advantages

- Less intrusive (only the markers are attached to the body)
- Very accurate
- Can capture not only the human (animal) motion, but also skin movements

## Shortcomings

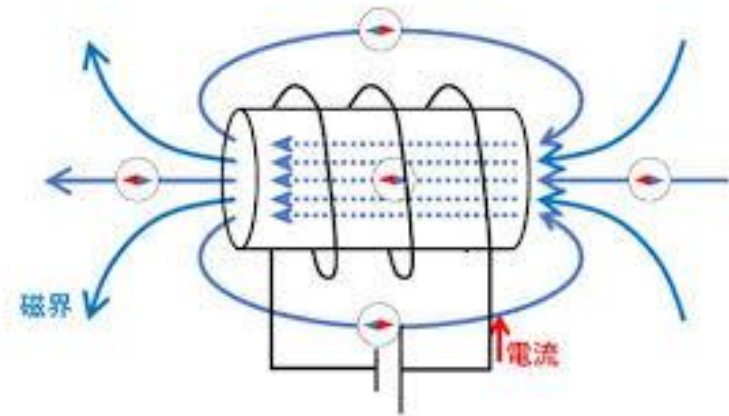
- Suffer from occlusions ( the body hiding the markers)
- A lot of post-processing is required if occlusion happens
  - Labelling which marker is which
- Need a large studio for a capturing outdoor movements

# Demo movie of an optical Mocap system



# Magnetic Mocaps

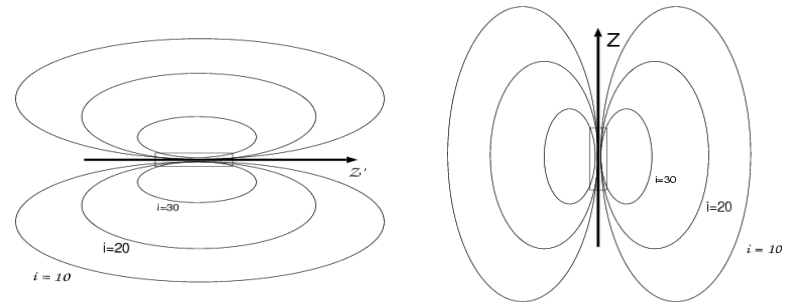
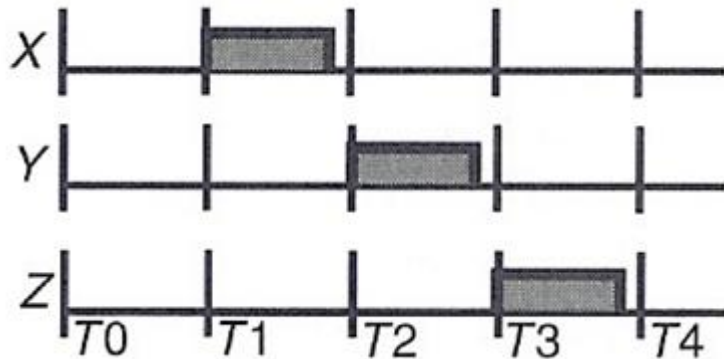
- Predicting the 3D pose using the magnetic field
- The magnetic field is constructed by a transmitter when electricity flows through the coil
- The strength of the magnetic field is detected by sensors attached to the body



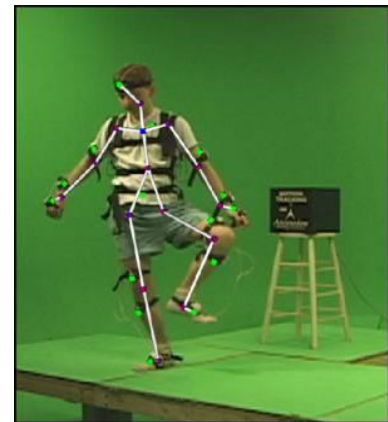
**André-Marie Ampère**

# Magnetic Mocaps

- The transmitter produces three orthogonal magnetic fields sequentially by the AC current

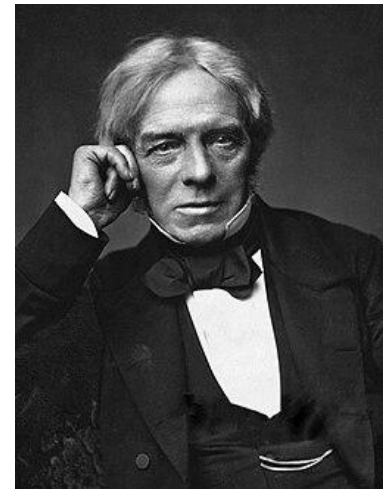
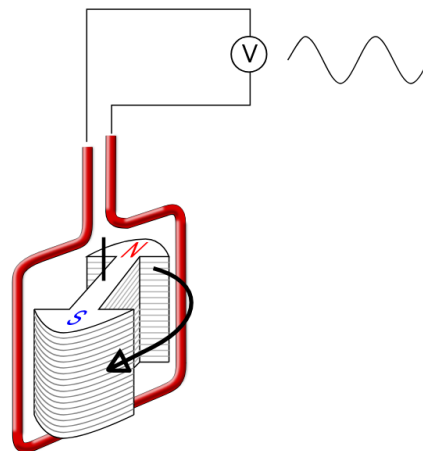


- The actor wears a suit to which the magnetometers are attached
- The magnetometers also made of coils.
- When the magnetic field changes electricity is produced and flows through the coils



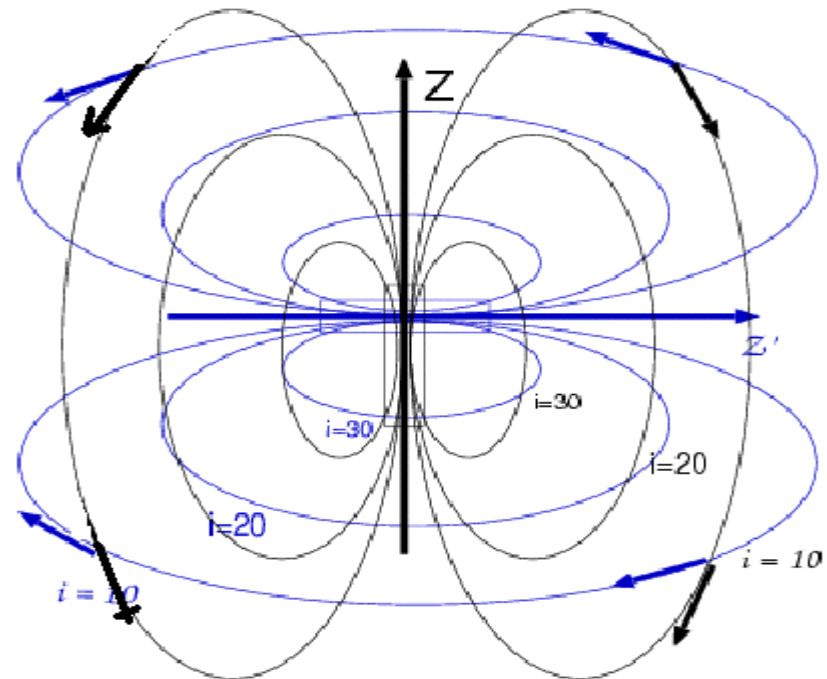
# Magnetic Mocaps

- The magnetometers are composed of three directions of coils. The magnetic field passes through the coils and then it produces electricity, which is detected.
- Based on the **Faraday's law of electromagnetic induction**, as the magnetic field passes through the coils and then electricity is produced.



# Magnetic Mocaps

- The farther you go away, the weaker the magnetic field is
- By using three magnetic fields, the position can be narrowed to one point



# Cons and Pros of Magnetic Mocaps

## Advantages

- We do not have to worry about occlusion
  - Motions of close contact can be captured
- No manual post-processing is required

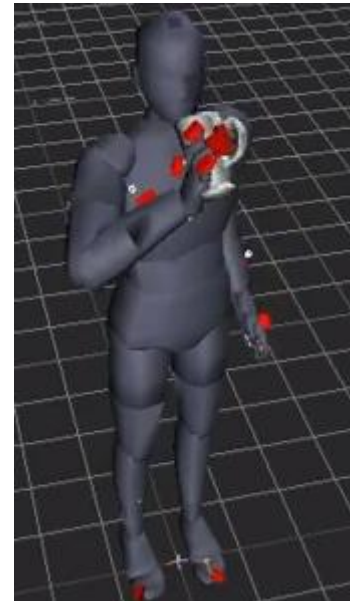
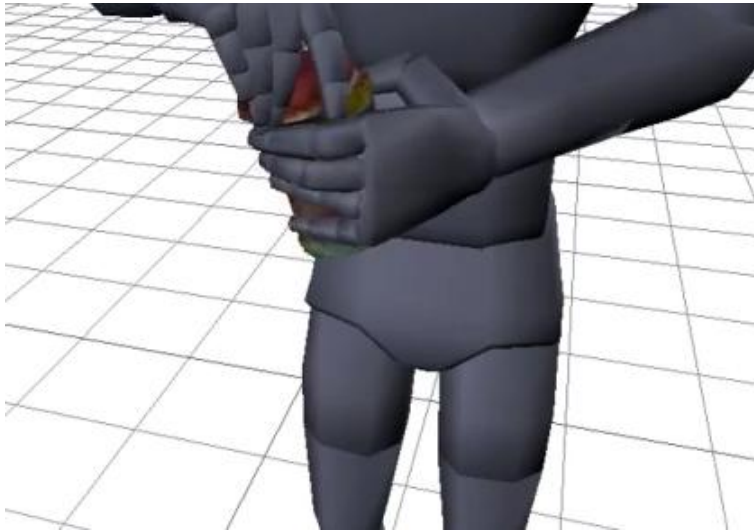
## Shortcomings

- Less accurate
  - The absolute positions are highly distorted
    - Easily affected by the noise
    - Cannot have metal/electronic devices in the capturing area
      - Affects the magnetic field
- The capturing volume is very small
  - Only 2-3 m away from the transmitter

# Edinburgh Interaction Database

By Peter Sandilands, Myung Geol Choi

- We captured the human movements as well as the objects manipulated
- Also captured the geometry of the objects



# Edinburgh Interaction Database: Motion Capture Using Magnetic Sensors and the Kinect

Peter Sandilands  
Myung Geol Choi  
Taku Komura



THE UNIVERSITY of EDINBURGH  
**informatics**

**ipab**

Institute of Perception,  
Action and Behaviour



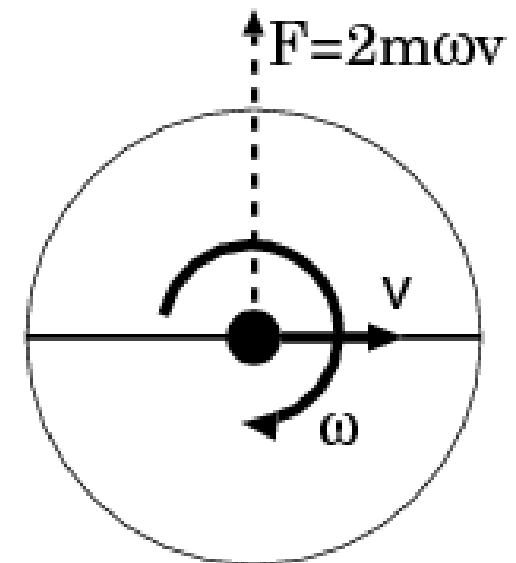
# Inertial trackers

- Inertial trackers measure the rate of change in
    - Angular velocity - gyro sensor
    - Translational acceleration - accelerometer
  - These values are integrated to compute the
    - Orientation and
    - Position
- of the tracker



# Orientation

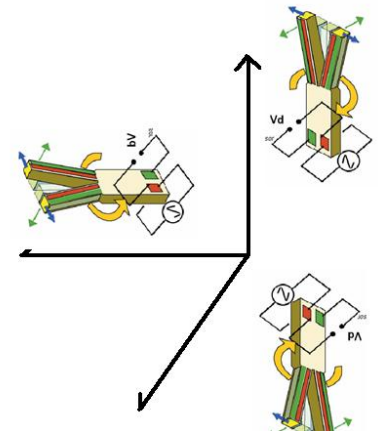
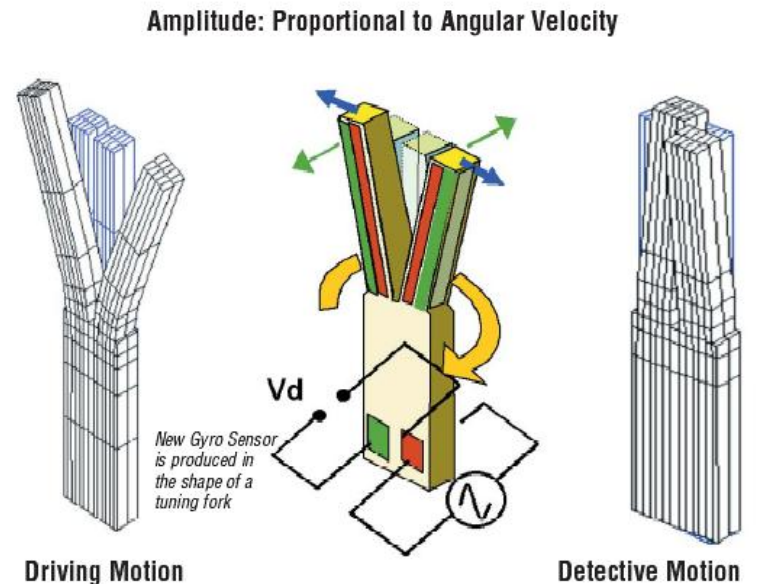
- The rate of change in object orientation or angular velocity, is measured by Coriolis-typed gyroscopes



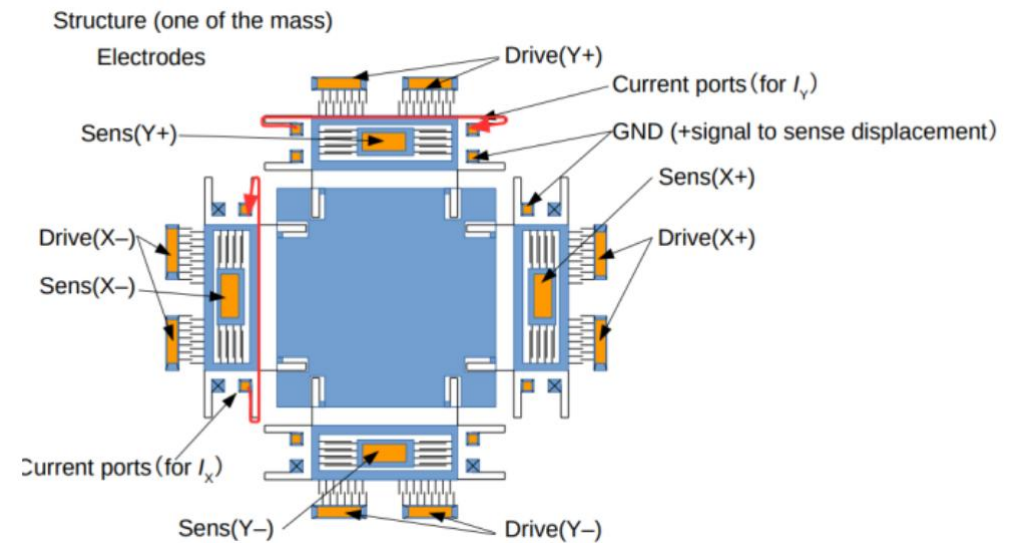
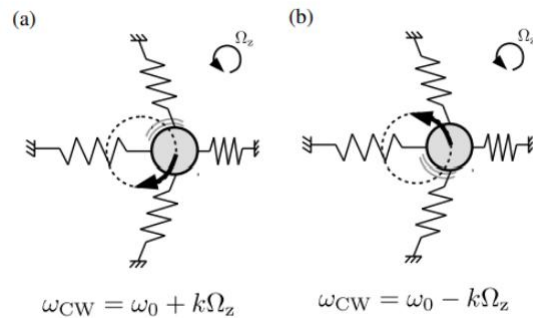
$\omega$ : angular velocity,  $v$ : velocity  
 $m$ : mass,  $F$ : coriolis force

# An example of a Gyro sensor

- Fujitsu Gyro Sensor
- The sensor is shaped like a tuning fork, and vibrates continuously.
- As the sensor turns, it is rotated and the Coriolis force affects it in the direction perpendicular to the vibration.
- this is converted into a proportional voltage, allowing the degree of rotation to be measured.
- **Amplitude: Proportional to Angular Velocity**



# Various types of mechanical structures

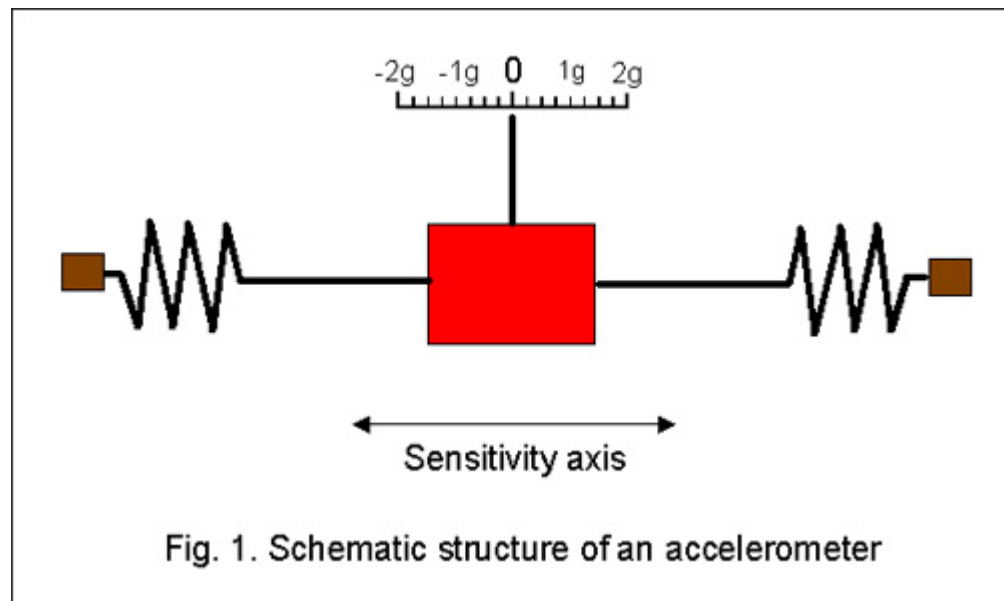


<https://marketing.ipros.jp/contents/basics/basic-gyro-sensor2/>

# Translation

- The translation velocity or acceleration is calculated using accelerometers
- Three accelerometers machined coaxially
- the position of the tracker is calculated by double integration of the acceleration

$$Ma = -k x$$



# CONS and PROS of inertia trackers

## Advantages

- Unlimited range of tracking
- No line-of-sight constraints

## Shortcomings

- Rapid accumulating errors, or drift
- Gyroscope bias leads to an *orientation error that increases proportionally with time* due to integration
- Accelerometer bias induces an error that *increases with the square of time*
- The only answer is to periodically *reset the error using other types of trackers*

Inbox (2,240) [Official Musi] MVN Anim X Download Yo Unity WebGL presentation Facebook Oculus Quest My World of rgbd\_motion +

← → ↺ ↻ https://www.xsens.com/products/mvn-animate?hsCtaTracking=0031f976-823a-4074-8cc4-d6f2347422ae584bb7ed- ☆

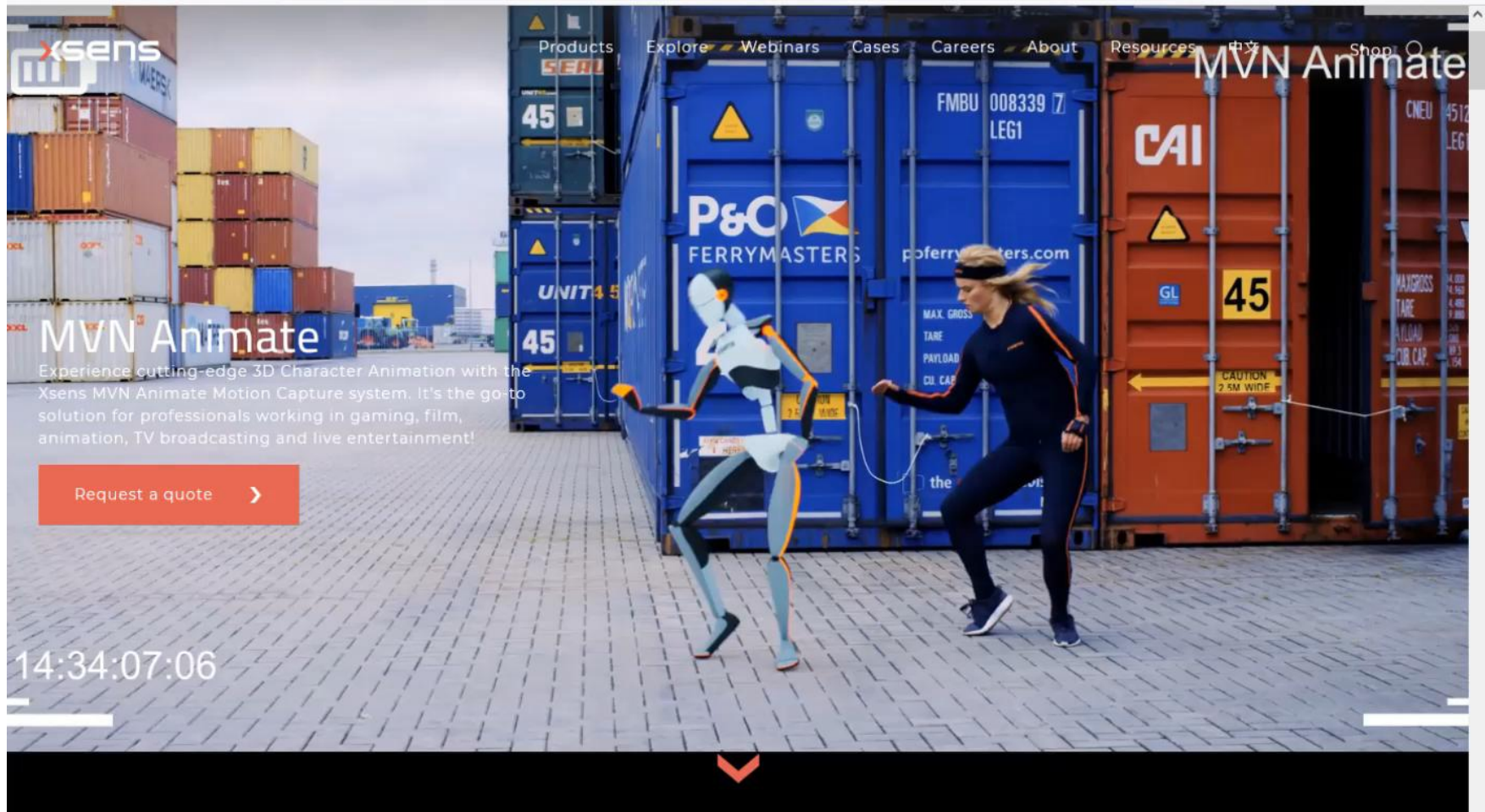
Products Explore Webinars Cases Careers About Resources 中文 Shop

# MVN Animate

Experience cutting-edge 3D Character Animation with the Xsens MVN Animate Motion Capture system. It's the go-to solution for professionals working in gaming, film, animation, TV broadcasting and live entertainment!

Request a quote >

14:34:07:06



### Xsens Cookie settings

By clicking "Accept All Cookies", you agree to the storing of cookies on your device to enhance site navigation, analyze site usage, and assist in our marketing efforts. [Cookie Policy](#)

Cookies Settings

Accept All Cookies



# Mechanical Mocaps

- Kinematic structure composed of links interconnected using sensorized joints
- The mechanical joints can directly measure the rotation of the human joints

## Advantages

- Very accurate
- Little latency
- Again, we don't have to worry about the occlusion
- Unlimited range of capturing

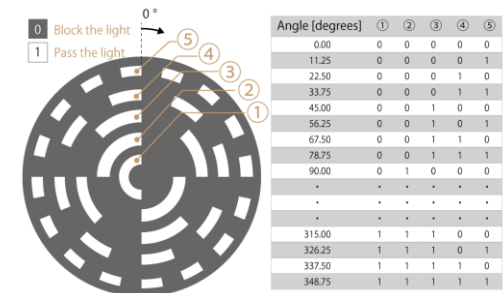
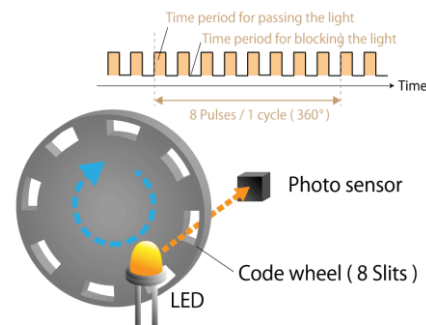
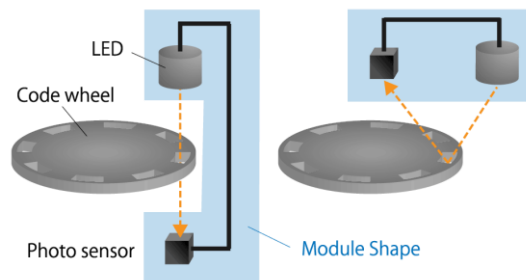
## Shortcomings

- More interference with the actor
  - Affects the performance of the actor
- Vulnerable
- Need an additional sensor to detect the location of the body root



# Optical Encoders

- Sensor to detect the angle/rotation
- Light is casted and detected at the photo sensor
- By measuring the number of pulses, we can detect the angular velocity, the number of rotations
- By using multiple slits, we can encode the absolute angle (useful for robotics, exoskeleton)



# Exoskeleton for Human Support

- Detect the human motion by the sensors and support the humans by providing external torque at the joints
- Used in the factories to support the back of human workers
- Also used for rehabilitation of injured people



# Video-based Motion Capture Systems

- Video-based systems capture the body motion from one or more RGB videos
- Using machine learning methods for 2D/3D pose prediction



OpenPose

# Training Data

## Body key-points dataset

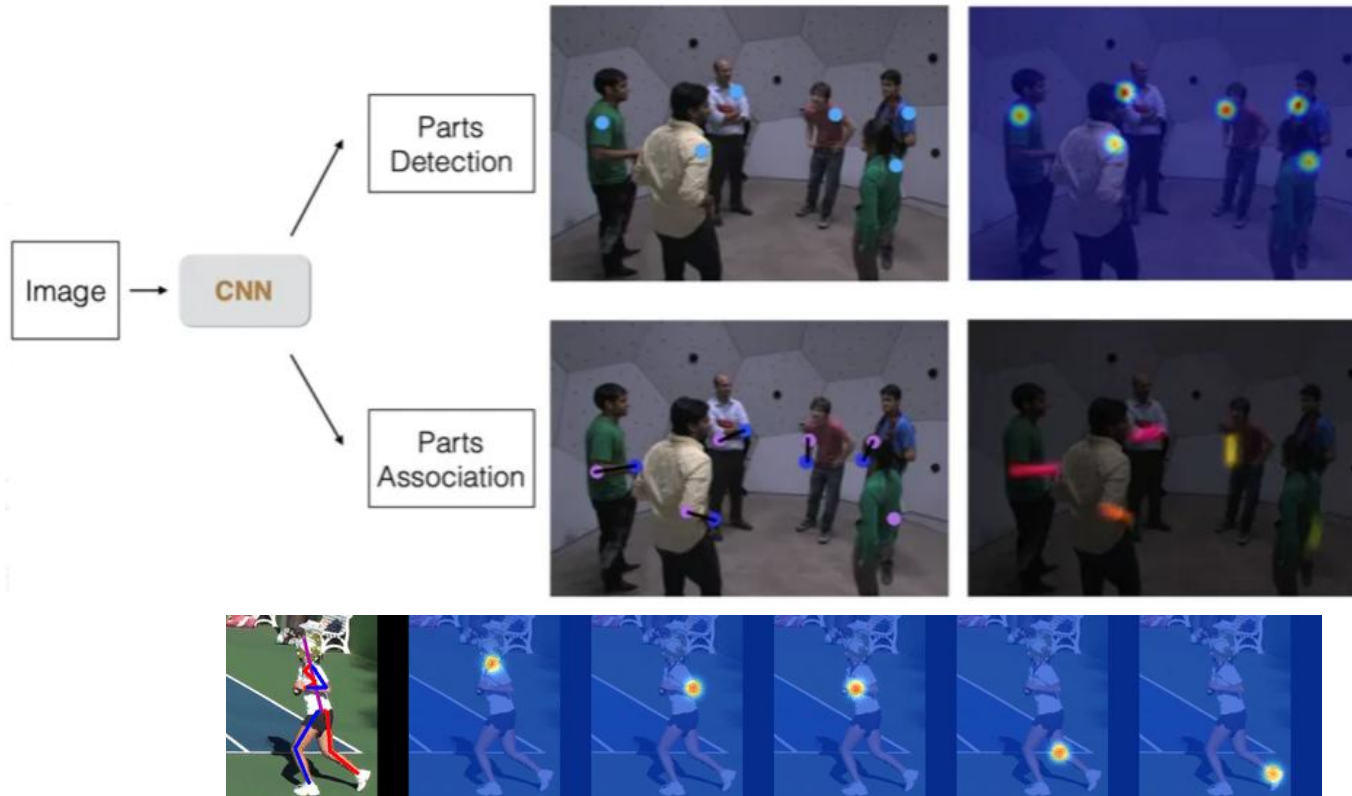


1. Labeled by real human
2. Huge data scale (For example, COCO-WholeBody has more than 200K image-label pair)
3. Train a deep learning model that predicts the joint position from the given image

*COCO-WholeBody, ECCV 2020*

# Video-based motion capture

## Jointly Learning Parts Detection and Parts Association



1. The network produces a heatmap of the joints
2. A confidence value also predicted



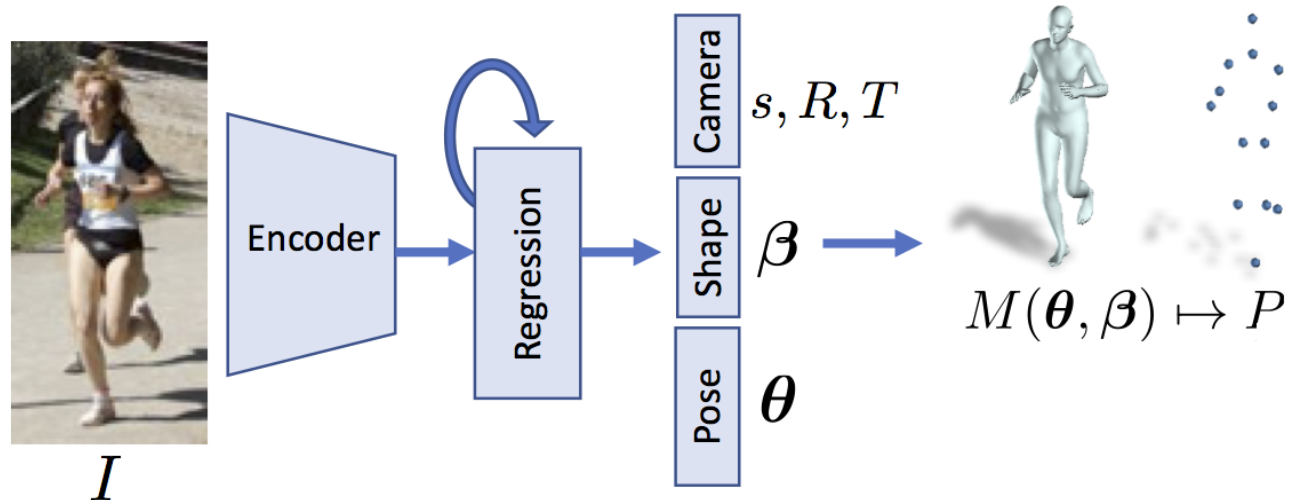
<https://www.youtube.com/watch?v=C1Sxk6zxWLM>

[CVPR paper video](#)

# **Real-time Multi-Person 2D Pose Estimation Using Part Affinity Fields**

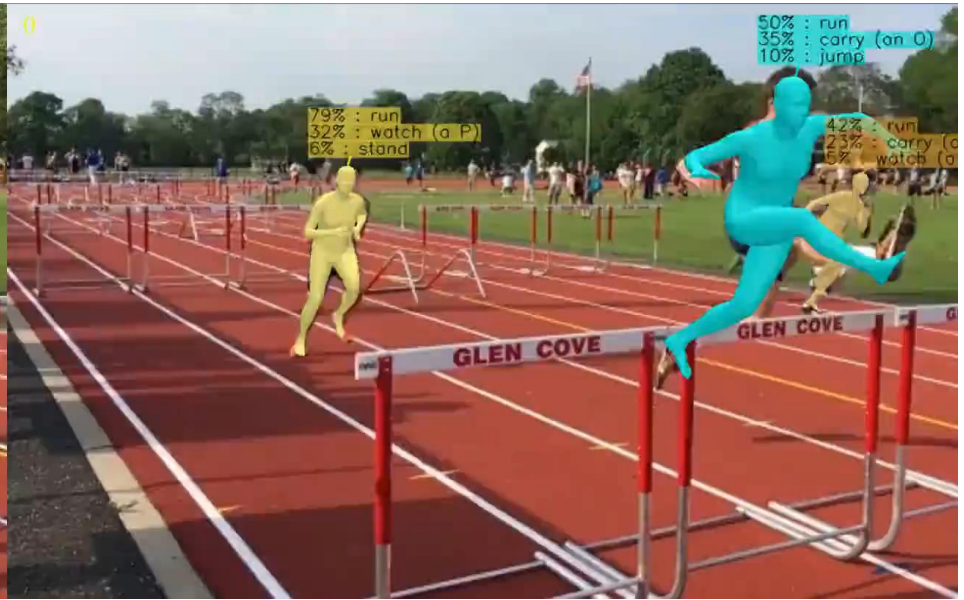
Zhe Cao, Tomas Simon, Shih-En Wei, Yaser Sheikh  
Carnegie Mellon University

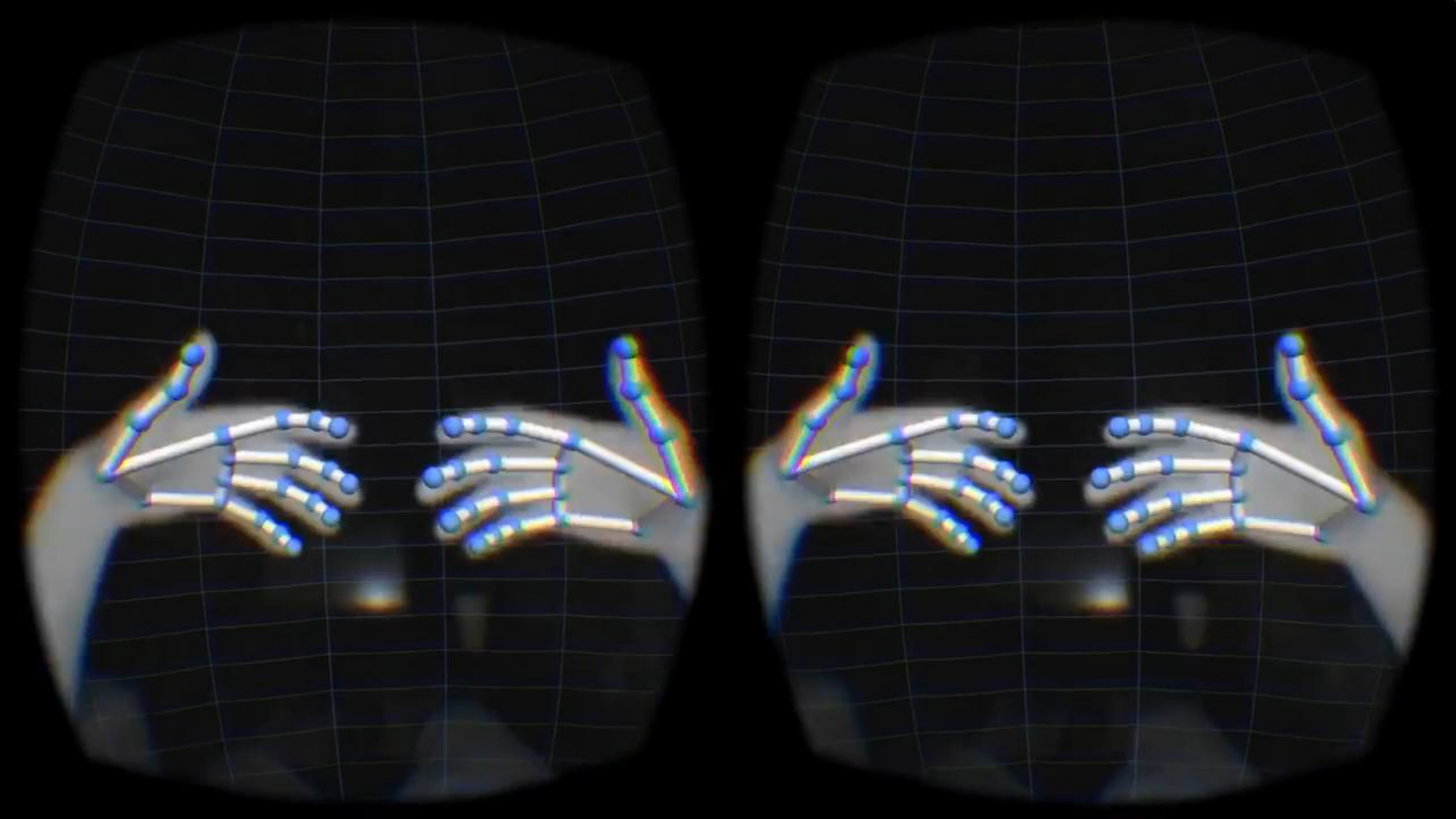
# Video-based motion capture: 2D to 3D



After predicting the 2D joint positions, the other parameters such as body shape, 3D pose, and camera parameters can be predicted

A 2D projected pose can be computed and a 2D loss can be computed



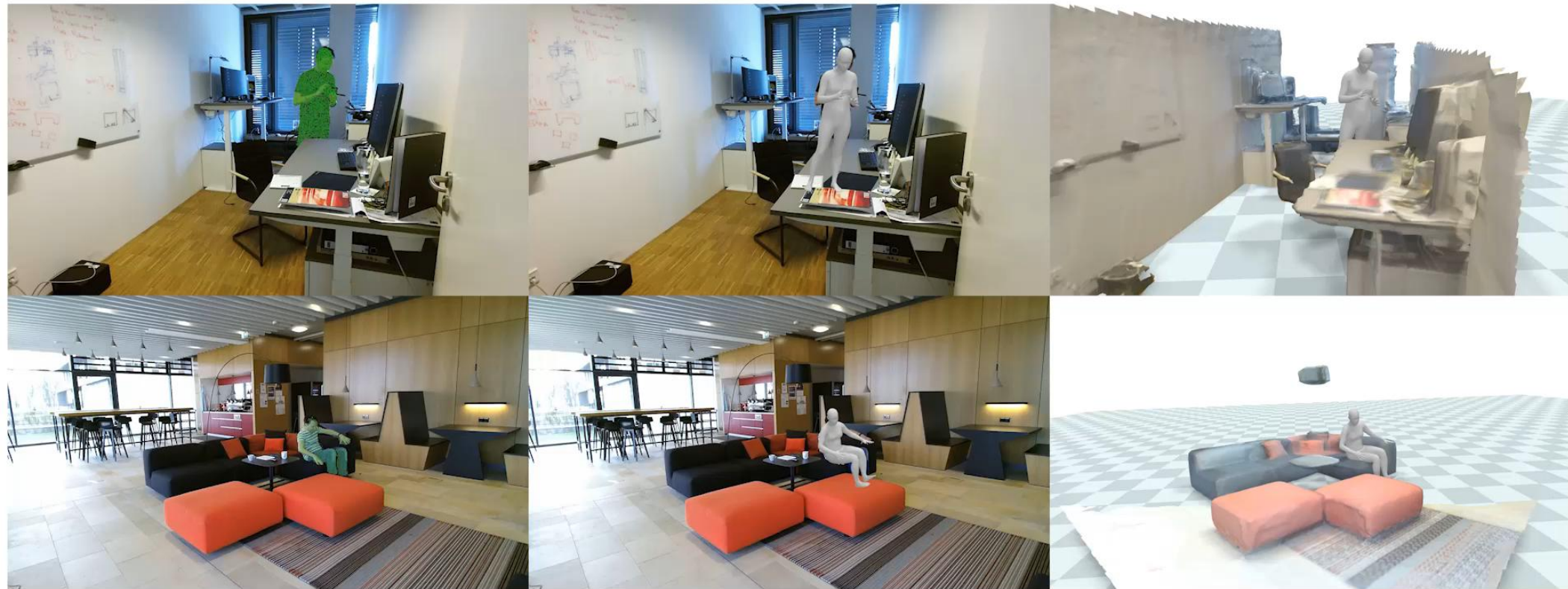


Oculus Quest

*Input*

*Motion & Shape*

*Ground Plane*



We can further consider a motion model trained with motion capture data to predict the poses while the body is occluded

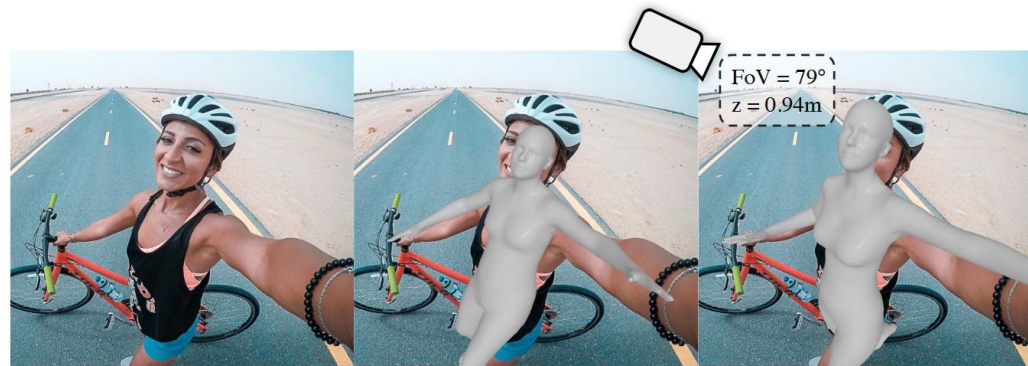
# Video-based Motion Capture Systems

## Advantages

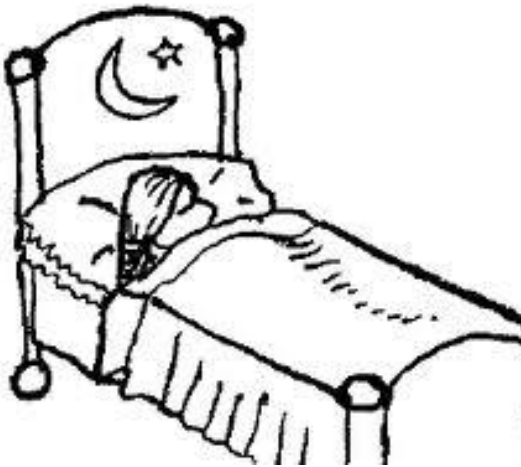
- Require no expensive devices
- No markers/sensors required

## Shortcomings

- Most methods cannot track the global body motion
- Accuracy still not high
- Many models assuming weak perspective projection



# Which mocap system will be good for the following motions?



# Summary

- Different mocap systems are appropriate for different movements
  - Optical mocap is the most popular system
  - Magnetic systems are good for capturing close interactions but the capture volume is small
  - Inertia-based systems show more flexibility for the users
  - Video-based motion capture is becoming better and better – can expect more improvement for practical usage soon.

# Character Animation

- There are three methods
  - Create them manually
  - Use real human motion
  - Use physically based simulation

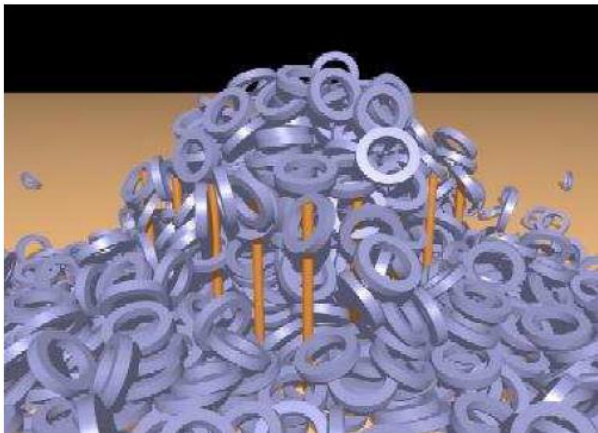


# Overview

- Physically-based animation
  - Using Newton Physics to simulate events
  - Applicable to human animation
- Methods based on
  - Forward dynamics
  - Inverse dynamics
- Forward dynamics – need to decide torque
  - PD control

# Physically-based animation

- Using Newton's laws of dynamics to simulate various phenomena
  - Drop objects in the scene, and let them collide and see what happens
  - Blowing hair by a dryer
  - Controlling bodies by force/torque



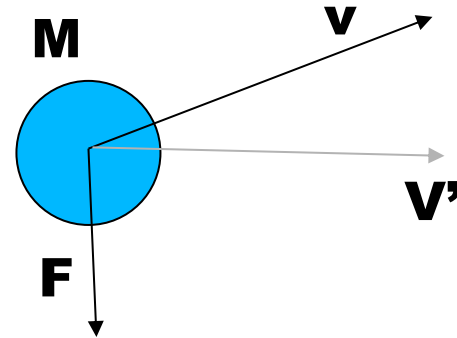
# Newton's Law of Dynamics

$$\mathbf{F} = \mathbf{M} \mathbf{a}$$

$\mathbf{F}$  : force

$\mathbf{M}$  : mass

$\mathbf{a}$  : acceleration



Given the force  $\mathbf{F}$ ,  
calculate the acceleration by  $\mathbf{a} = \mathbf{F} / \mathbf{M}$   
then integrate

$$\mathbf{v} = \mathbf{v}' + \mathbf{a}\Delta t$$

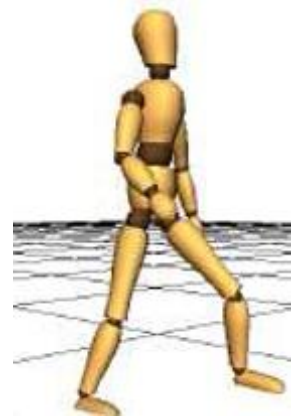
$$\mathbf{x} = \mathbf{x}' + \mathbf{v}\Delta t$$

# Newton's Law of Dynamics

- For multi-body objects like human bodies, it is a bit more complex

$$\boldsymbol{\tau} = \mathbf{H} \ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) + \boldsymbol{\tau}_g$$

$\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}$ : generalized coordinates, velocity, acceleration



$\boldsymbol{\tau}$  : force, or torque made at the joints

- $\mathbf{H}$  : Mass matrix,  $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$  Coriolis force
- $\boldsymbol{\tau}_g$  : gravity

# Forward vs. Inverse Dynamics

## Inverse Dynamics

The calculation of forces given a set of accelerations

## Forward Dynamics

The calculation of accelerations given a set of forces


$$\tau = H \ddot{q} + C(q, \dot{q}) + \tau_g$$

# Using Forward Dynamics for Human Animation:

**Problem:** How do we decide the internal force?

- *Must decide how the muscles exert force*
  - *Passive motions*
    - *Falling down (No force or torque, rag-doll)*
    - *pushed and stepping back*
  - *Voluntary motions → Walking, Running*



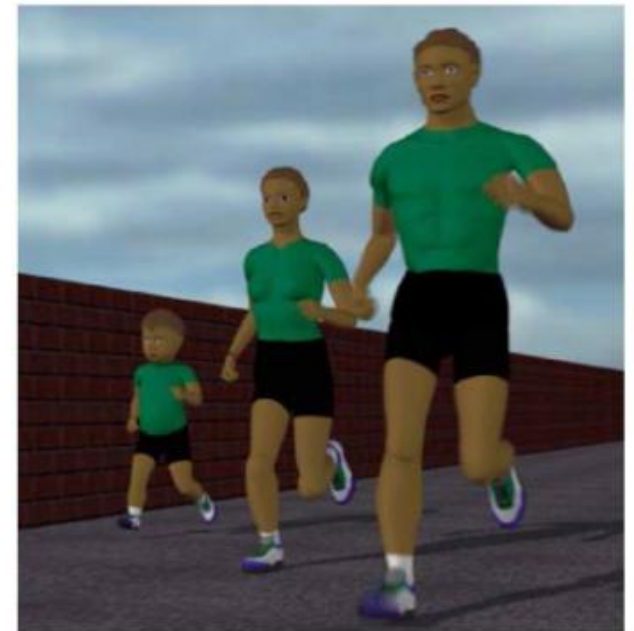
# Passive motions - *Rag doll physics*

- *The body just falls down powerlessly*
- *No active force exerted by the muscles*
- *Some passive force*
  - *elastic force by the ligaments and muscles*



# Voluntary motions

- Motions like walking, running, reaching
- Humans control the body by exerting force by the muscles
- To simulate such motions, need to determine the torque
  - PD control
  - More complex control methods (optimal control, evolutionary control, reinforcement learning etc.)



# PD control : Introduction

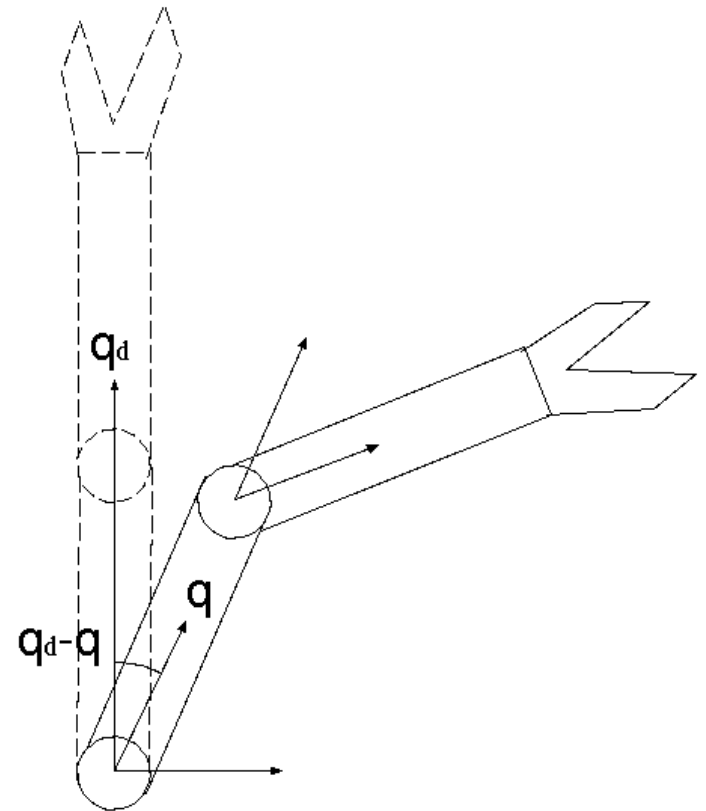
- A method used in robotics
- The larger the difference between the current state and the desired state, the larger the force/torque is

$$\tau = a(\mathbf{q} - \mathbf{q}_d) + c(\dot{\mathbf{q}} - \dot{\mathbf{q}}_d)$$

$a, c$  : constants,

$\mathbf{q}, \dot{\mathbf{q}}$  : current state/velocity

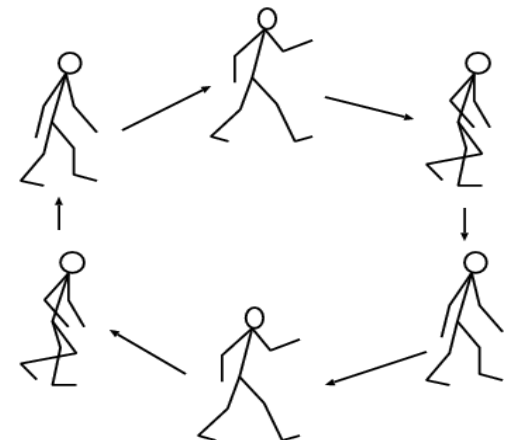
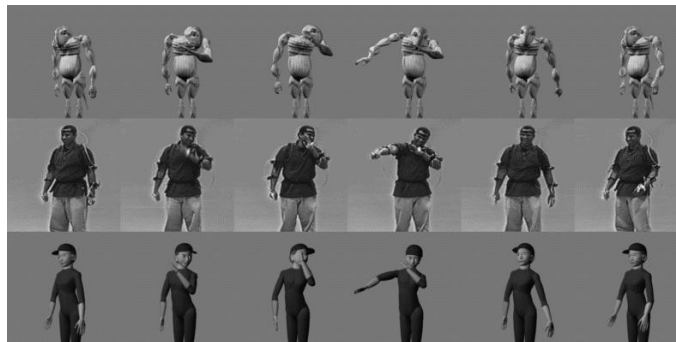
$\mathbf{q}_d, \dot{\mathbf{q}}_d$  : desired state/velocity



# Voluntary motions by PD control

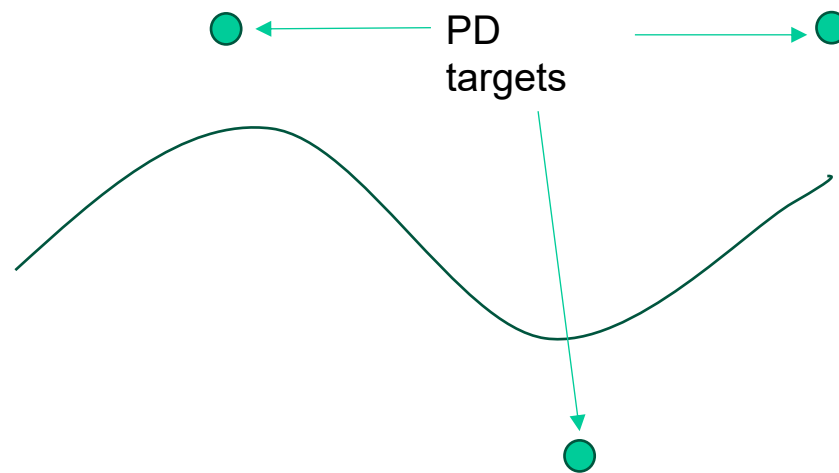
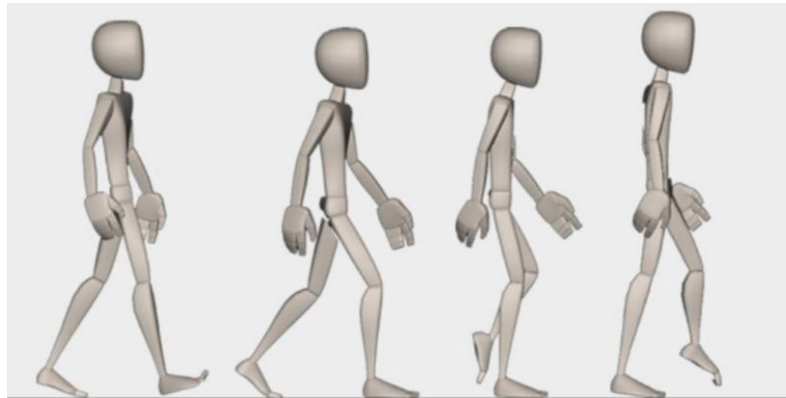
- Prepare a number of keyframes
- Compare the current state of the avatar and the keyframes (desired state)
- Apply torque proportional to the difference of the current state and the target state

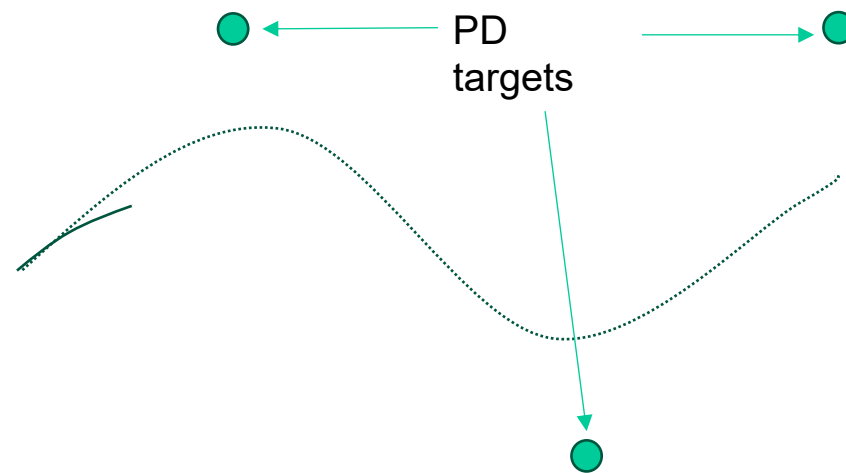
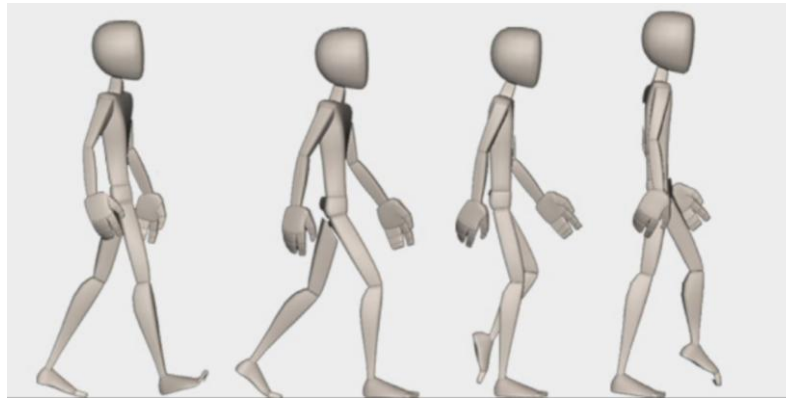
Switch the desired state once the body is close enough to it

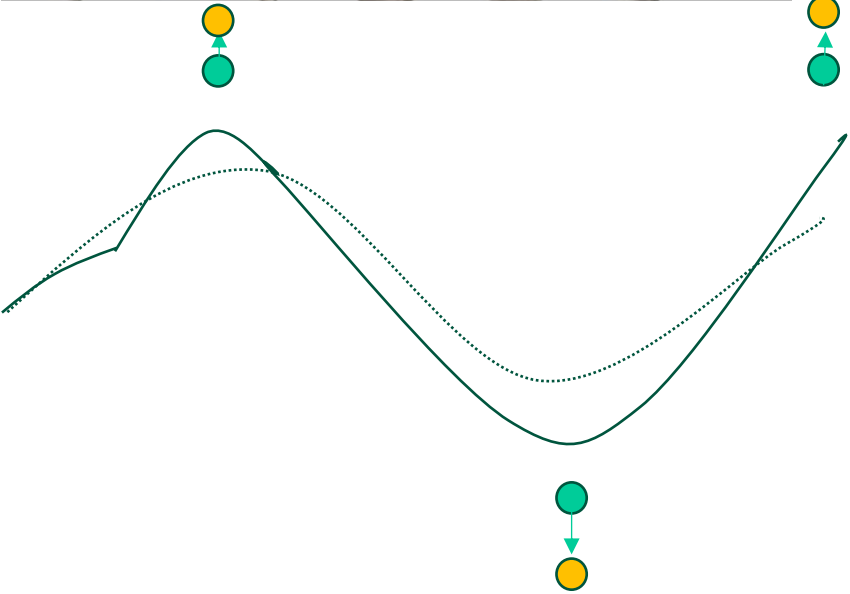
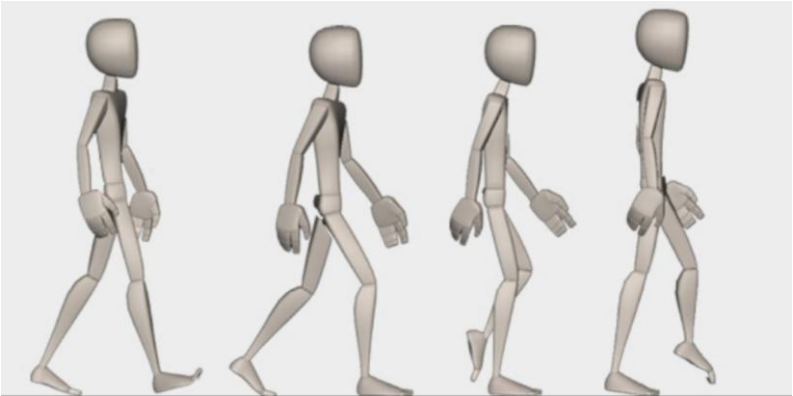


PD Targets: Previously, they were almost manually tuned...



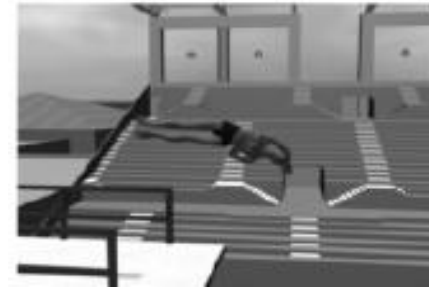






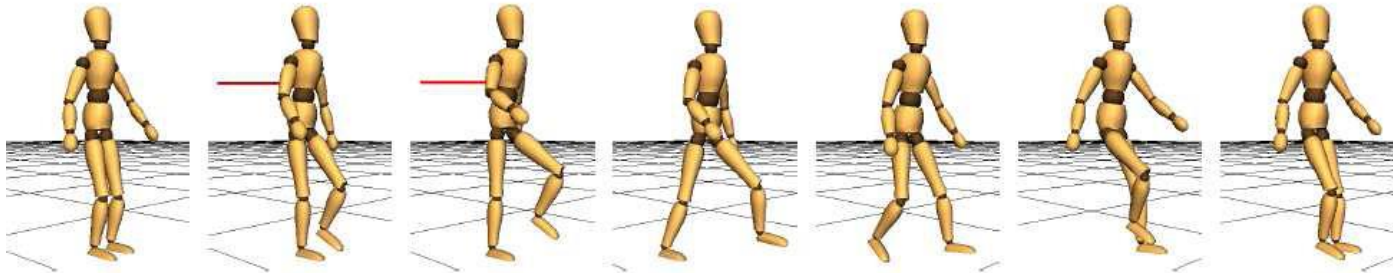
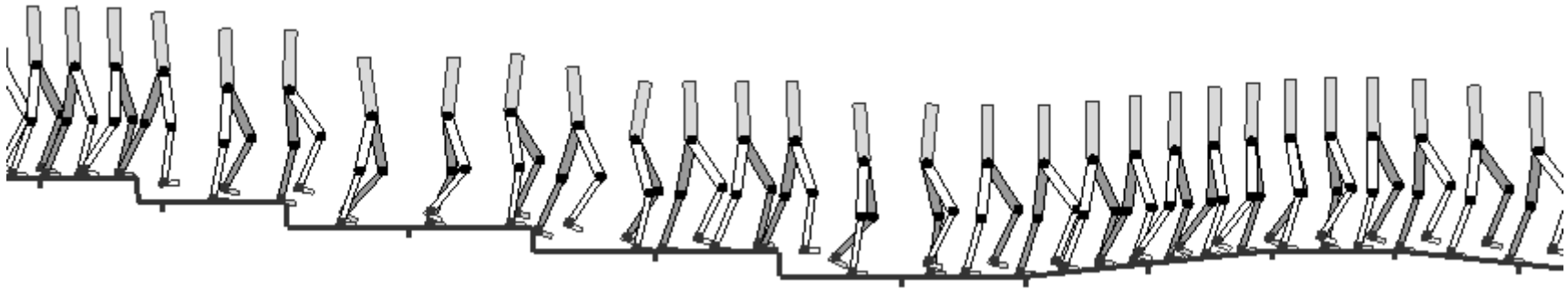
# Simulating Athlete's Motions

- Diving
- Running
- Cycling

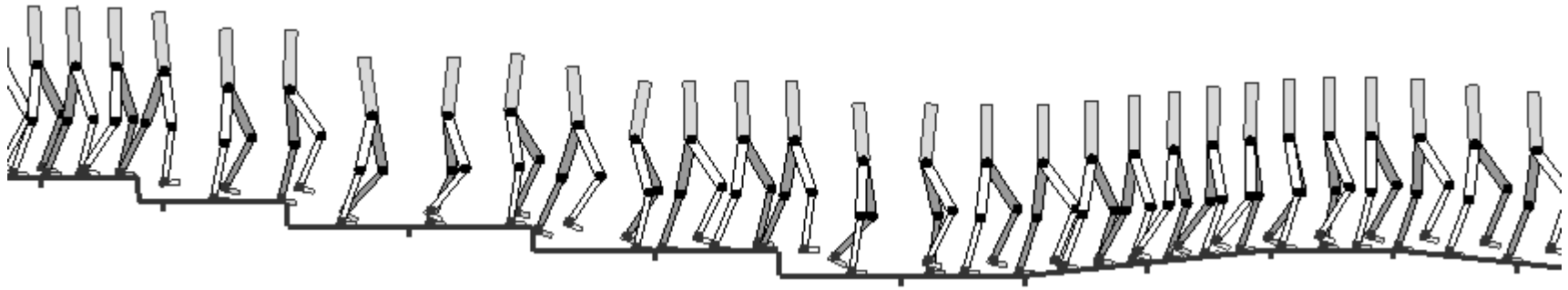


# SIMBICON (Yin et al. 2007)

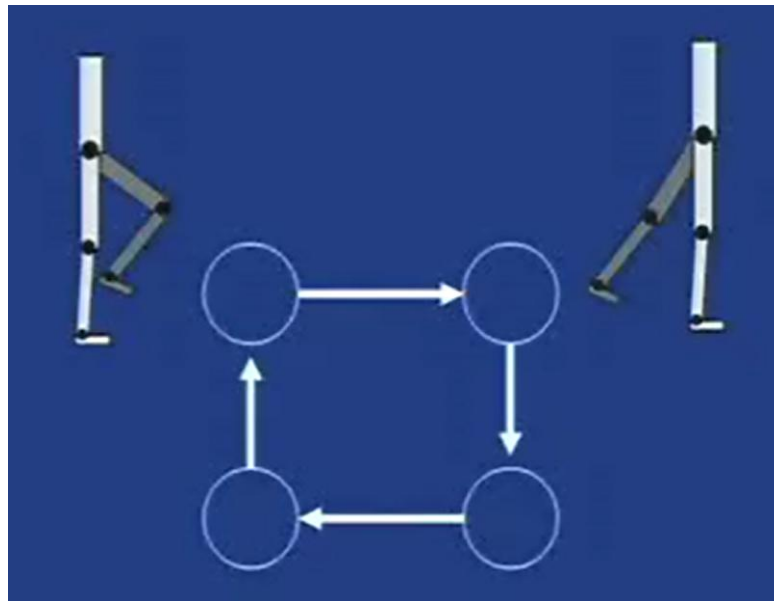
- A good PD controller-based biped locomotion controller
- Extra ideas for keeping the body well balanced



# SIMBICON (Yin et al. 2007)



- PD targets: the keyframes for computing the torques: 4 targets for walking, 2 for running

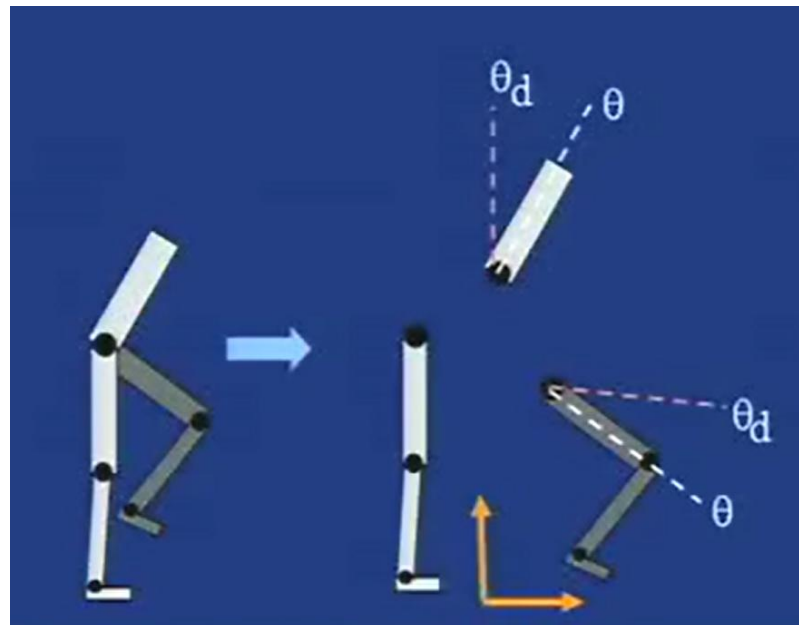


# SIMBICON

Tip 1.

Some PD targets in the world space

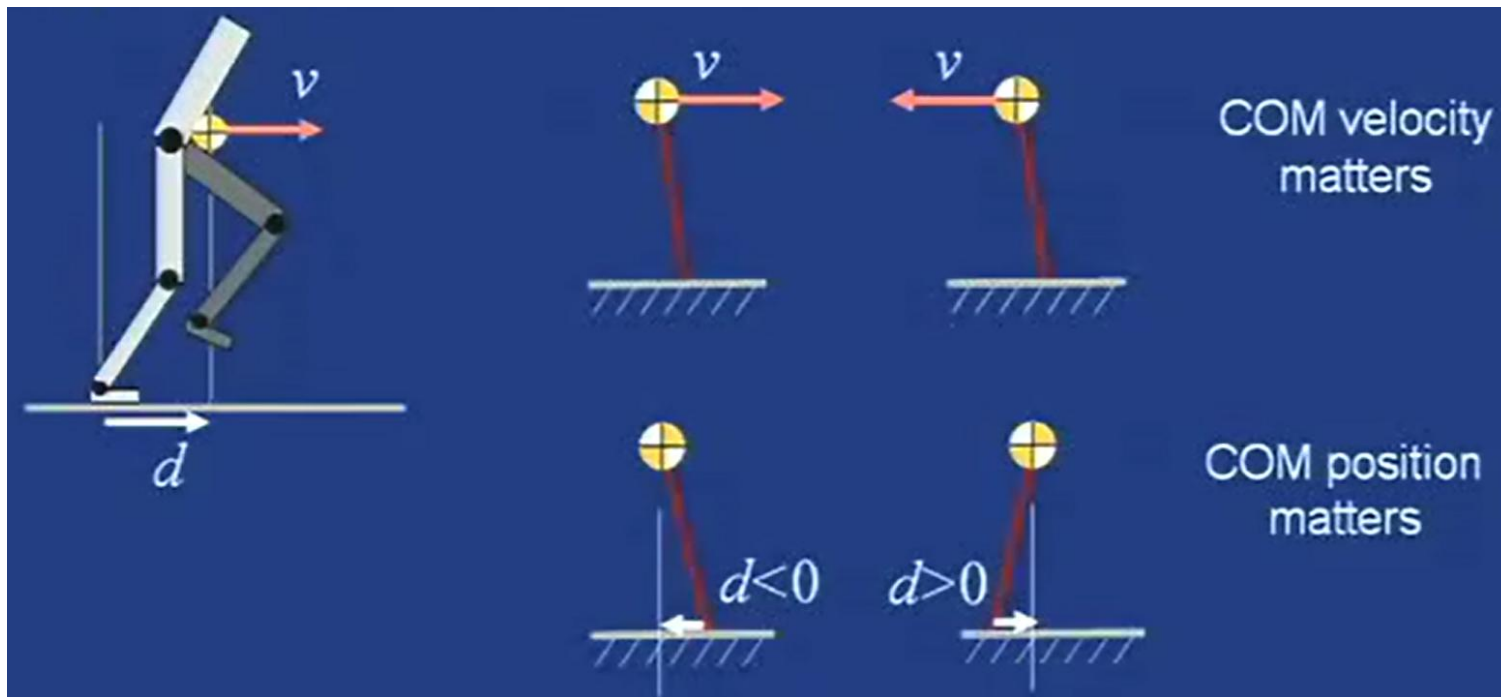
- The upright orientation of the torso
- The swing hip



# SIMBICON

## Tip 2.

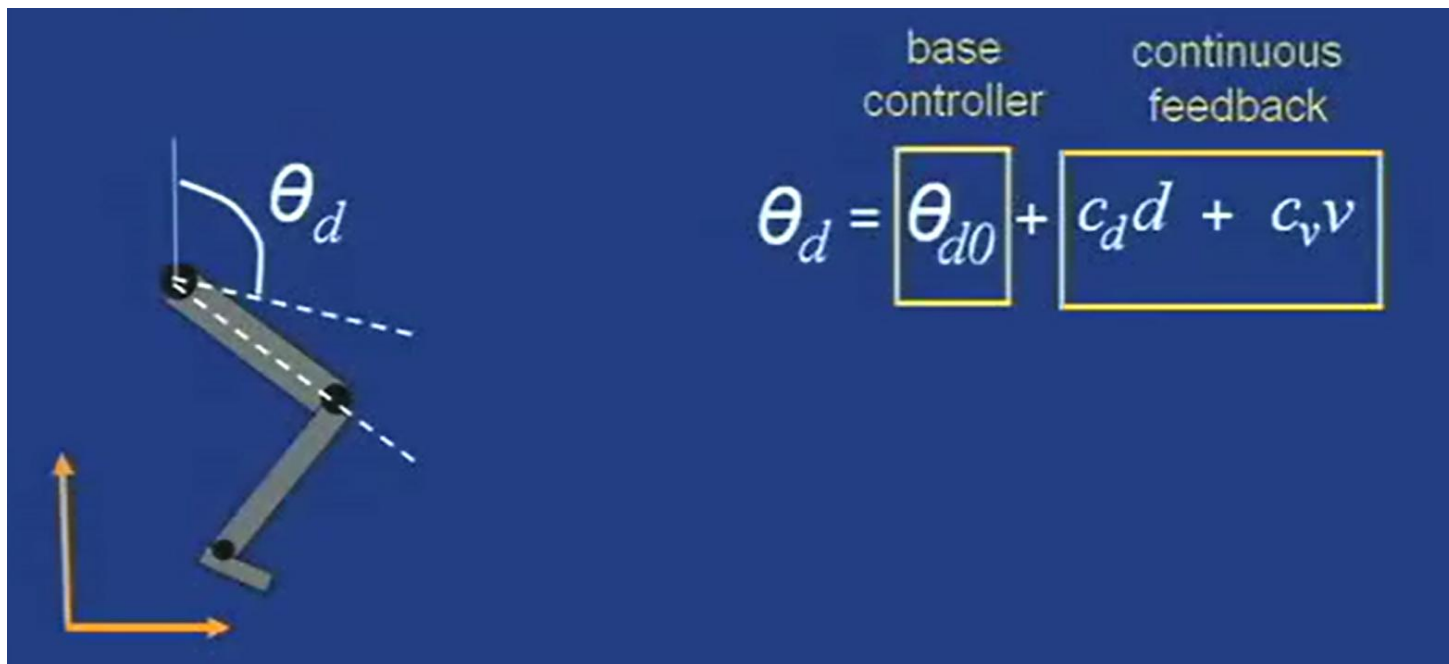
Adjusting the stepping leg/supporting leg according to the center of mass



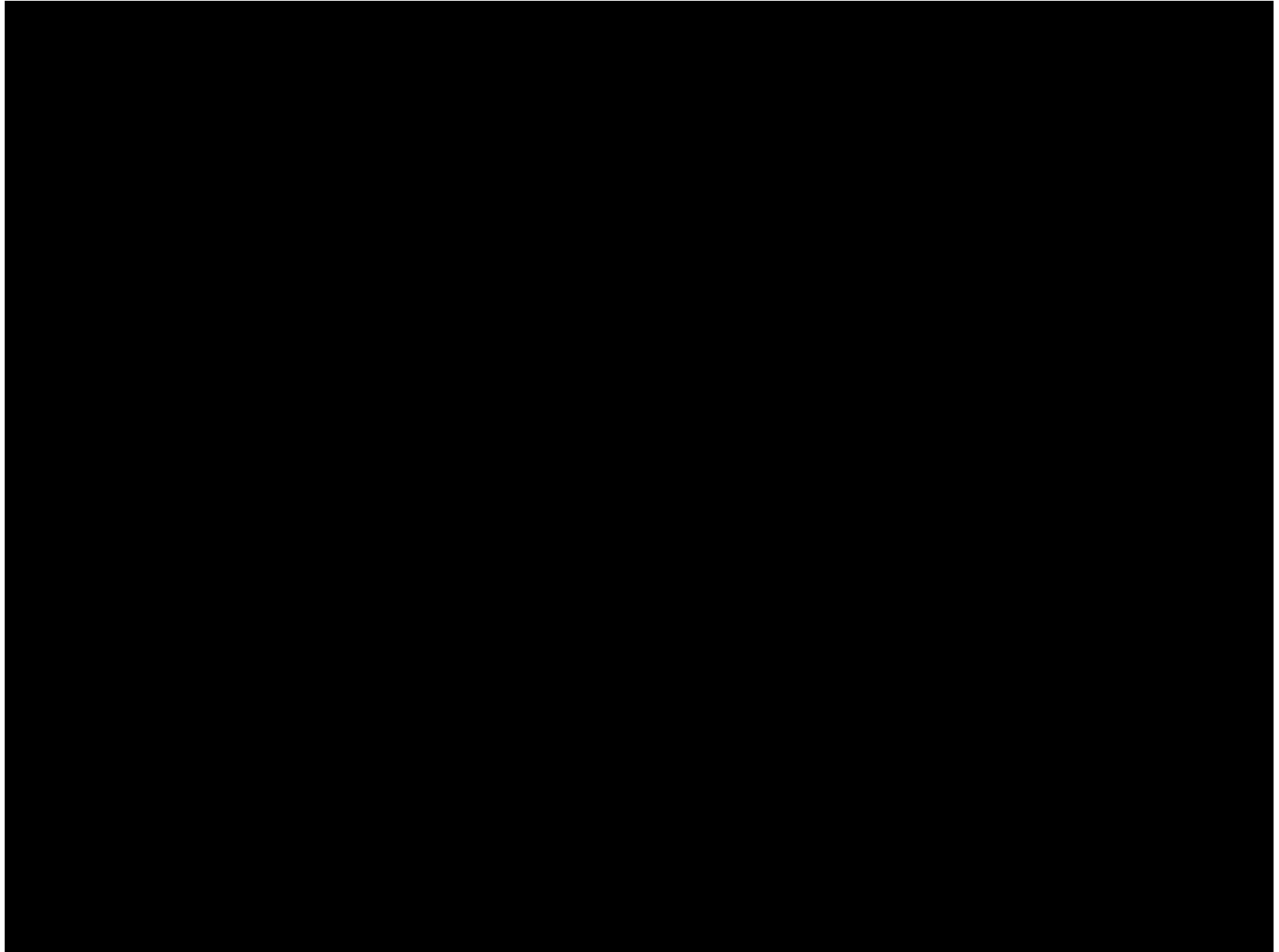
# SIMBICON

Tip 2.

Adjusting the stepping leg/supporting leg according to the center of mass

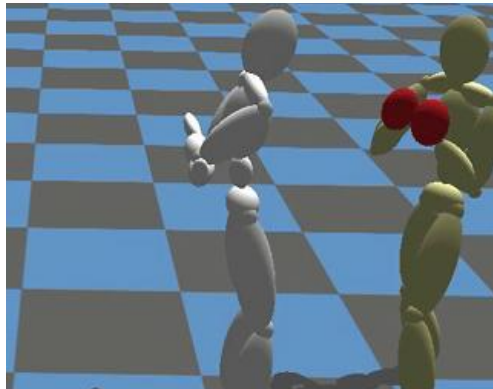


# SIMBICON : Demo video



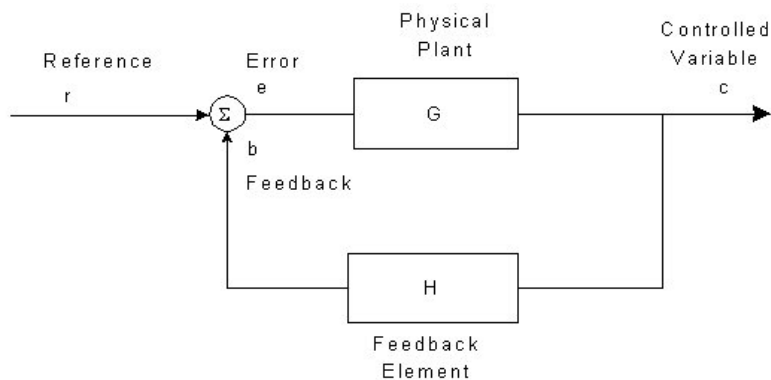
# Response Motion by PD control

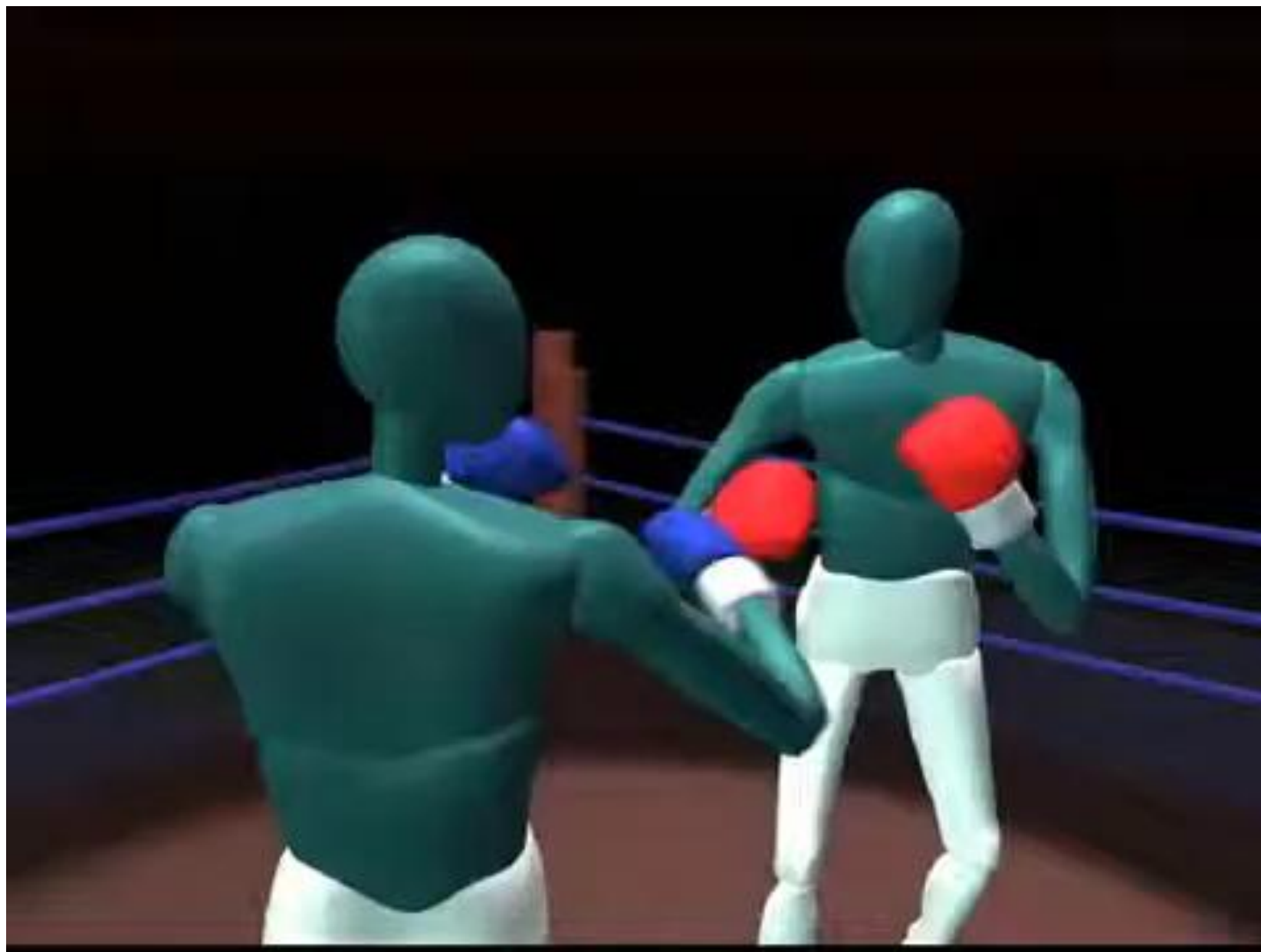
- Moving back to the original motion when perturbed
  - Feedback control
- Keeping balance or falling back



# Feedback Control

- Moving back to the original posture / motion when perturbed





# PD control : Difficulty

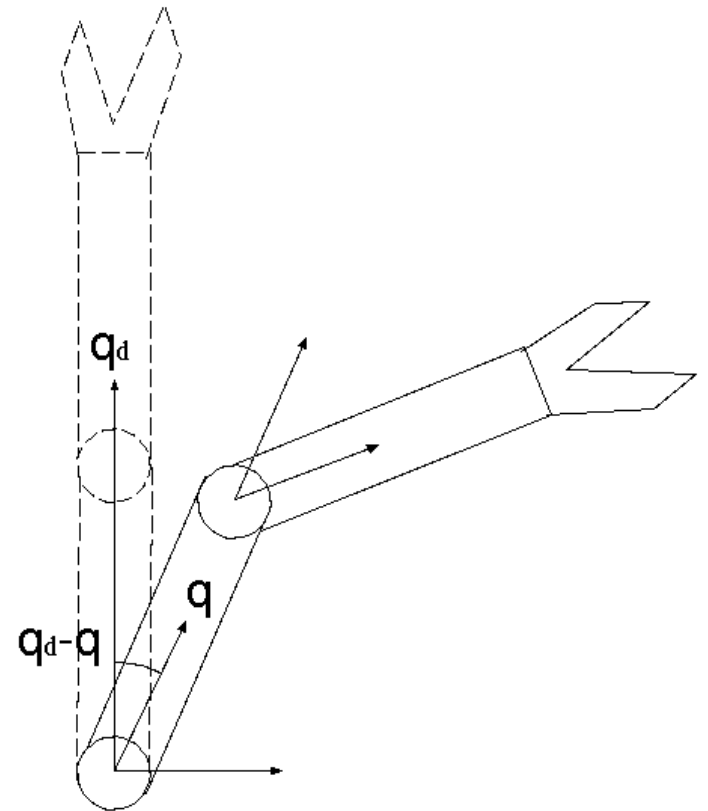
- Parameter tuning not easy

$$\tau = a(\mathbf{q} - \mathbf{q}_d) + c(\dot{\mathbf{q}} - \dot{\mathbf{q}}_d)$$

$a, c$  : constants,

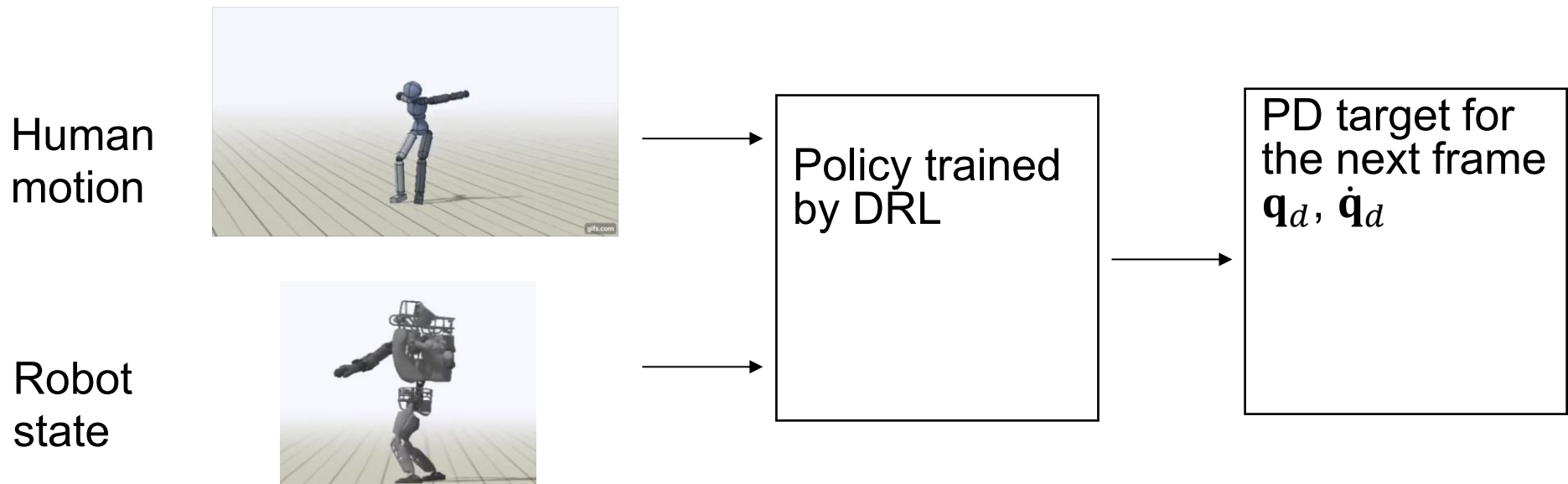
$\mathbf{q}, \dot{\mathbf{q}}$  : current state/velocity

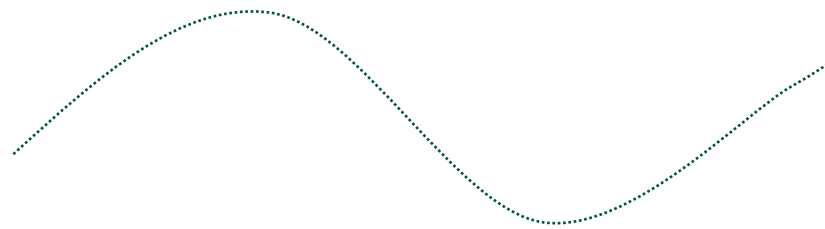
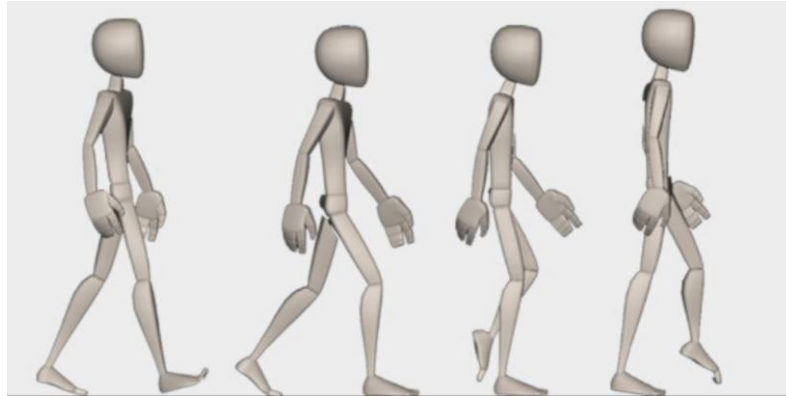
$\mathbf{q}_d, \dot{\mathbf{q}}_d$  : desired state/velocity



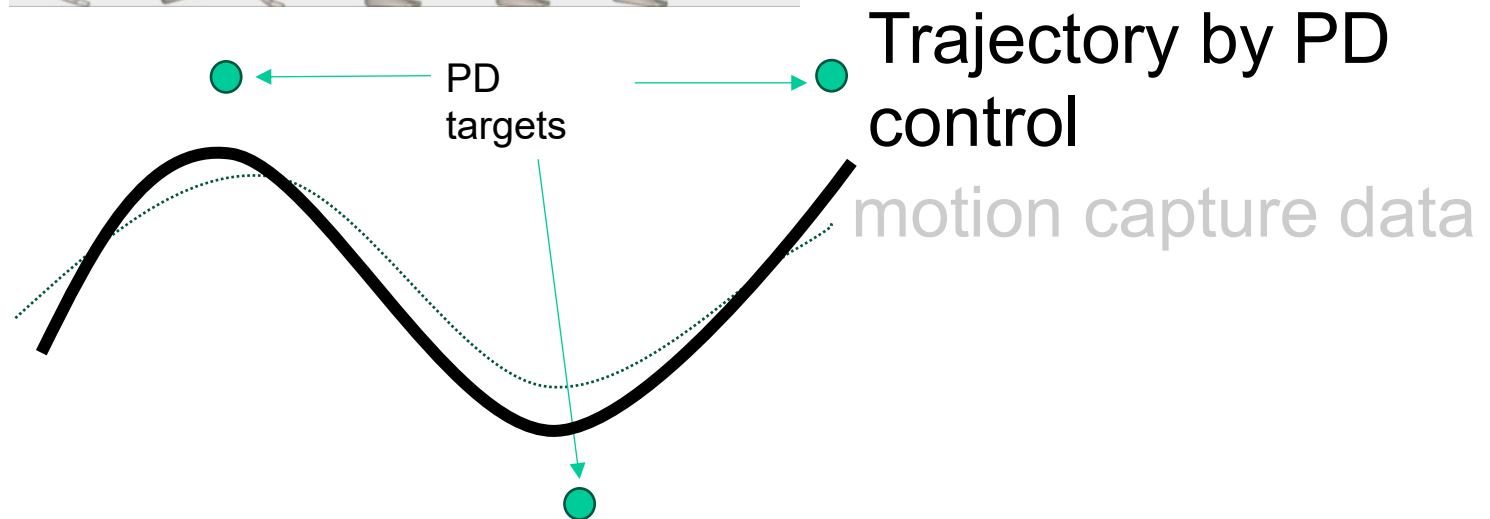
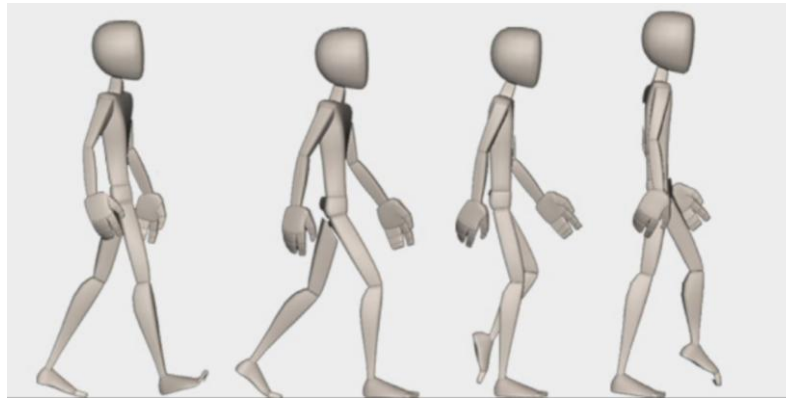
# Deep Reinforcement Learning for Tracking Human Motion

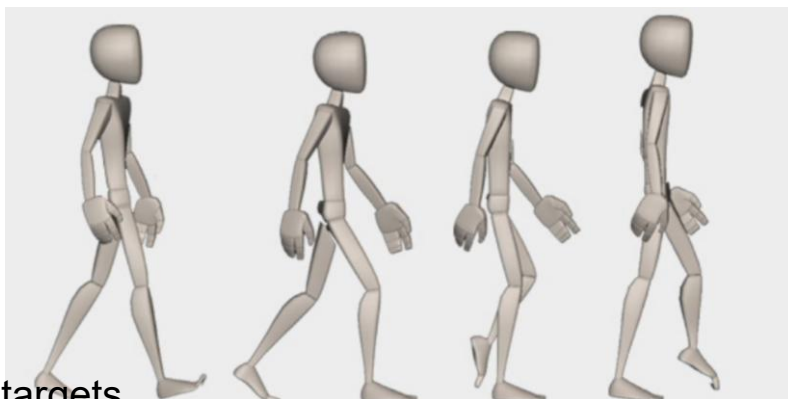
- PD control is also used for humanoid robots to mimic human motion.
- A policy learned by deep reinforcement learning computes the PD targets for mimicking the motion capture data





motion capture data





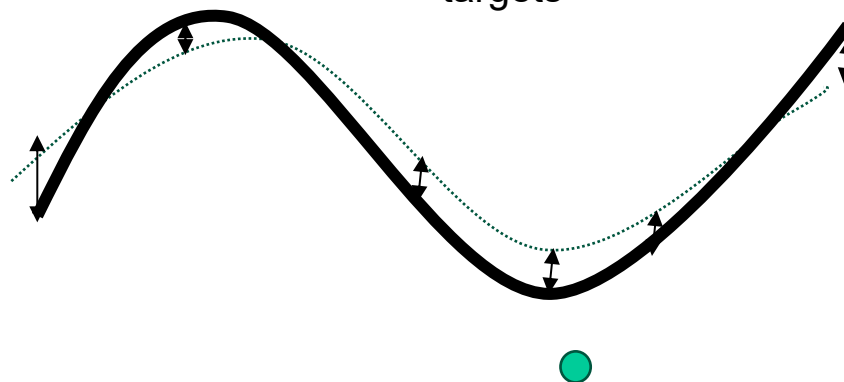
PD targets



PD  
targets



Trajectory by PD  
control  
motion capture data



# Reinforcement Learning

- Reinforcement learning (RL) solves a sequential decision-making problem
- Given the current state  $s_0$ , a policy is trained such that it selects a series of actions  $a_0, a_1, \dots$ , such that the accumulated reward defined as follows is maximized:

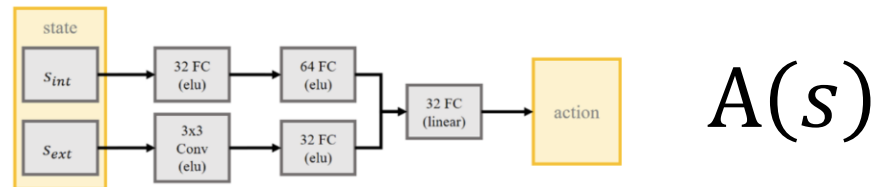
$$\eta(\pi) = E_{s_0, a_0, \dots} \left[ \sum_{t=0}^{\infty} \gamma^t r_t \right]$$

where  $r_t$  is a reward obtained at step  $t$  and  $\gamma$  is a discount factor which prevents the sum of rewards from becoming infinity

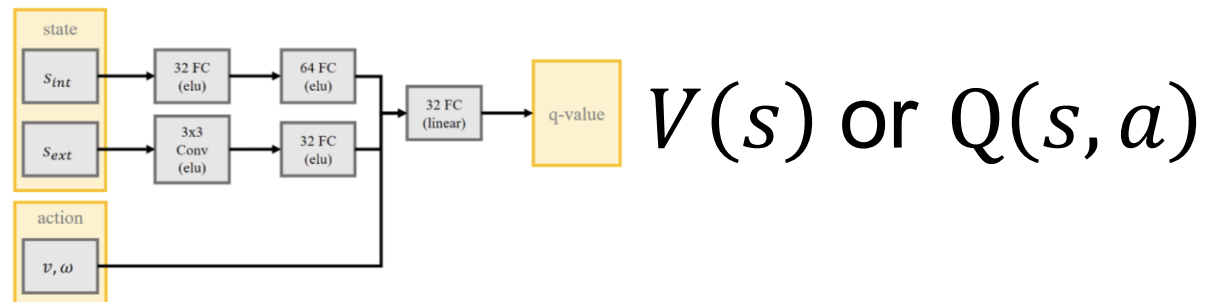
# Learning the Policy

- We can utilize deep reinforcement learning methods where the policy is learned by neural networks.
- More scalable and can generalize the training data

Actor: decides the action



Critic: computes the score



$A(s)$ : The action that should be taken at state  $s$  – that results in the best value

$V(s)$ : The value function,

$Q(s, a)$  : The Q function, the value you get when selecting action  $a$  at  $s$

# Training the Critic (Value Function)

$V(s)$  is trained by MSE loss

Compute  $V(s)$

Select an action by the Actor  $A(s)$  and then  
find the next state  $s'$

Compute the new value for  $s$ :  $V' = r + \gamma V(s')$

Train the critic with MSE:  $\|V(s) - V'\|$

## Training the Actor

- Using  $V(s)$  compute the value at the current state
- Select an action by the Actor  $A(s)$  and then find the next state  $s'$
- Compute the value at the next state for  $s'$ :
- Compute the advantage:  $A(s,a) = r + \gamma V(s') - V(s)$
- The advantage describes how good taking action  $a$  is than expected

## Training the Actor

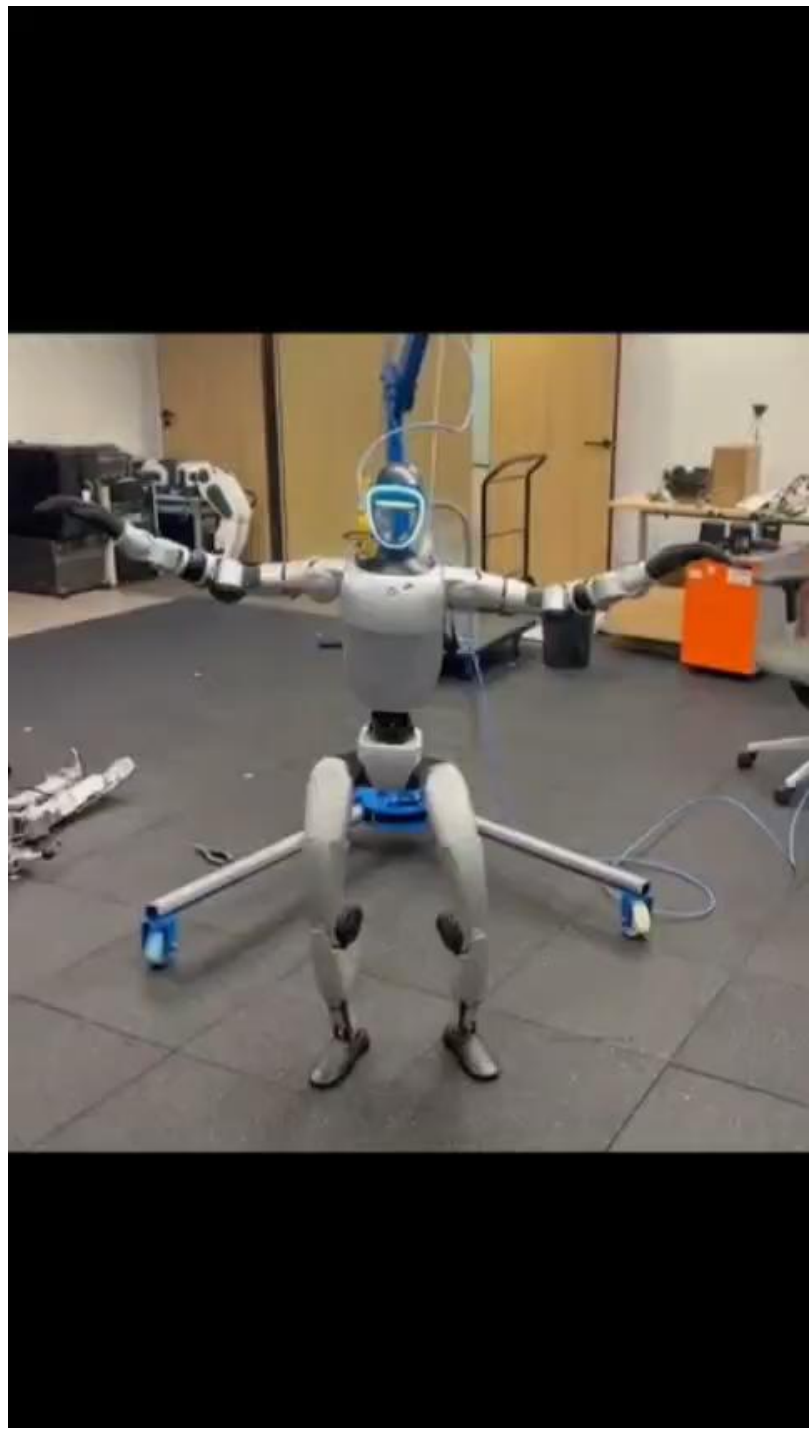
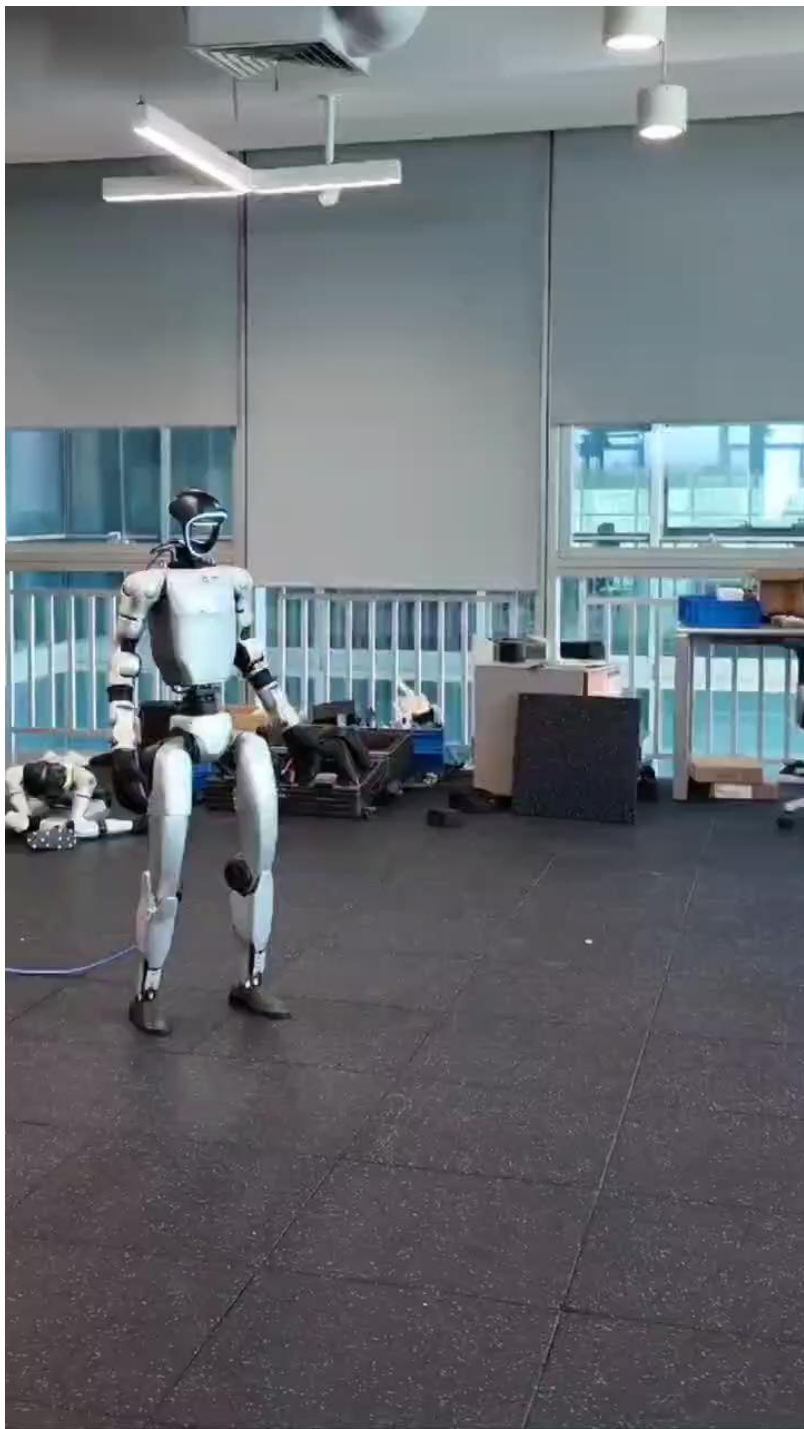
- $A(s,a) > 0$  : It is better than expected, so increase the probability to select action  $a$
- $A(s,a) < 0$  : It is worse than expected, so decrease the probability to select action  $a$

## Policy Gradient with Advantage:

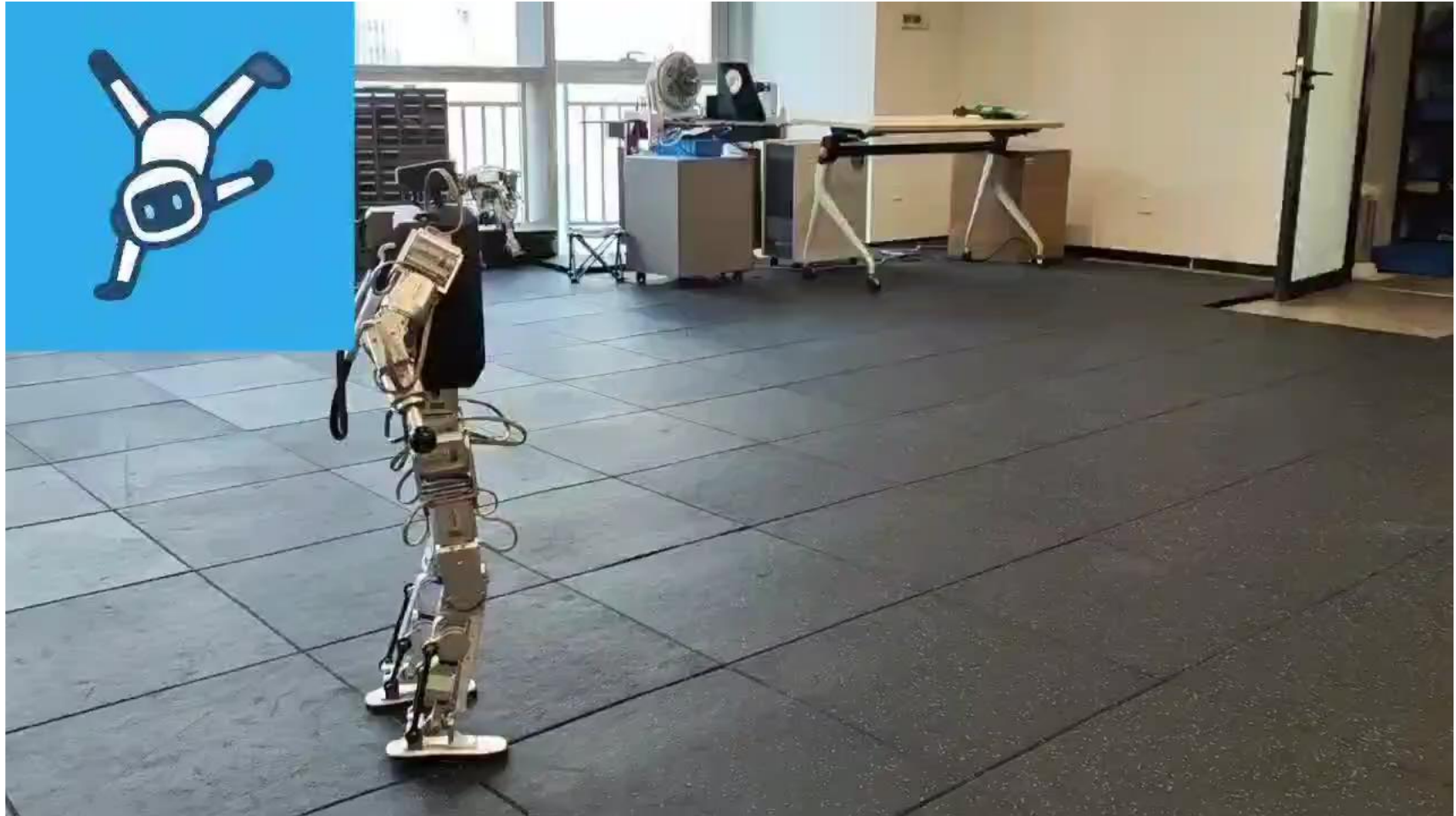
$$\nabla J(\theta) = \mathbb{E} [\nabla_{\theta} \log \pi_{\theta}(a|s) \cdot A(s, a)]$$

## Policy update:

$$\theta \leftarrow \theta + \alpha \cdot \nabla_{\theta} \log \pi_{\theta}(a|s) \cdot A(s, a)$$







## Quick Quiz again

Which method to solve the following problem?

- Find how fast you can break dance?
- Find how much forces are made by your muscles when punching?
- Control the muscles to move like Mike Tyson
- Produce an animation of falling down
- Produce an animation of hopping



# Readings

Skeletal Parameter Estimation from Optical Motion Capture Data

Adam G. Kirk et al.

IEEE Conf. on Computer Vision and Pattern Recognition (CVPR) 2005.

**Automatic Joint Parameter Estimation from Magnetic Motion Capture Data** James F. O'Brien

Robert E. Bodenheimer, Jr. Gabriel J. Brostow Jessica K. Hodgins, GI2001

<http://www.cc.gatech.edu/gvu/animation/Papers/obrien:2000:AJP.pdf>

**Motion capture**

**Sang Il Park, and Jessica K. Hodgins:** [Capturing and Animating Skin Deformation in Human Motion](#), *ACM Transactions on Graphics*, 25(3): 881-889 (2006)

Angjoo Kanazawa – video-based mocap <https://people.eecs.berkeley.edu/~kanazawa/>

Response motion

- Zordan, V. B., Hodgins, J. K., [Motion capture-driven simulations that hit and react](#), ACM SIGGRAPH/Eurographics Symposium on Computer Animation, 2002, pp. 89-96.

- Zordan, V. B., Majkowska, A., Chiu, B., Fast, M., [Dynamic Response for Motion Capture Animation](#) , ACM SIGGRAPH 2005.

- [Pushing People Around](#) Okan Arikan David Forsyth James O'Brien  
Symposium on Computer Animation (SCA) 2005

Locomotion

Raibert, Hodgins, Animation of Dynamic Legged Locomotion, SIGGRAPH'91

Jack Wang et al. [Optimizing Walking Controllers for Uncertain Inputs and Environments](#), SIGGRAPH 2010

SIMBICON: <https://www.cs.ubc.ca/~van/papers/Simbicon.htm>